XTART

FORMACIÓN PROFESIONAL EN TECNOLOGÍA

XTECH

CICLO FORMATIVO DE GRADO SUPERIOR EN DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Berta Díaz Flores

Alberto Ordiales García

Carlos Corchado Benito

CURSO 2026-2027

TÍTULO: Mercury

AUTOR/AUTORES: Berta Díaz Flores, Alberto Ordiales García y Carlos Corchado Benito

TITULACIÓN: Desarrollo de Aplicaciones Multiplataforma

FECHA: 13/11/2026

RESUMEN

El proyecto ha consistido en diseñar, desarrollar e implantar una solución digital integral y coherente para Sneaker Factory, conectando su presencia web, la gestión empresarial y el procesamiento técnico auxiliar bajo un mismo modelo de datos y caso de uso. A nivel de interfaz, se ha construido una web corporativa y un catálogo de productos estándar con diseño responsive adaptado a móviles y escritorio, incorporando un apartado específico para el servicio Custom, se ha realizado una página de zapatillas personalizadas (definiendo modelo, colores y tallas). En el núcleo de negocio, se ha desplegado y documentado un entorno operativo con Ubuntu y Odoo, configurando la estructura de la empresa, los roles de usuario y el ciclo empresarial completo, desde el registro de clientes a oportunidades en CRM. De forma complementaria y sin interferir en las funciones propias del ERP, se ha desarrollado en Java un subsistema auxiliar de consola basado en programación multiproceso (ProcessBuilder), donde un proceso coordinador recibe las exportaciones de pedidos Custom (CSV/JSON) y gestiona procesos hijos especializados mediante comunicación por flujos estándar (stdin/stdout). Este subsistema valida la integridad de los datos, agrupa las cantidades por tallas y variantes, y genera automáticamente fichas auxiliares de taller, controlando en todo momento los tiempos de espera (waitFor), los códigos de salida y el registro de errores (stderr). Finalmente, se ha garantizado la trazabilidad de extremo a extremo mediante un caso de uso completo que recorre todo el sistema con datos compatibles, respaldado por un repositorio Git limpio y libre de credenciales, datos de prueba suficientes y una documentación técnica y de demostración totalmente reproducible

Enlace de GITHUB: https://github.com/Reckles1010/SneakerFactory

Cita – Frase célebre / Dedicatoria

"Cuando no se puede saber todo, hay que seguir adelante con lo que se tiene y rogar porque sea suficiente"

Índice

Capítulo 1. Resumen del Proyecto. 6
1.1. Contexto y justificación 6
1.2. Planteamiento del problema 6
1.3. Objetivos del proyecto 6
1.4. Resultados obtenidos 7
Capítulo 2. Desarrollo de Interfaces. 8
2.1. Planificación y metodología. 9
2.2. Identidad visual. 9
2.2.1. Variables de diseño :root. 12
2.2.2. Tipografía. 12
2.3. Estructura y arquitectura. 13
2.3.1. Estructura de ficheros. 13
2.4. Descripción detallada de las páginas. 14
2.5. JavaScript. 17
2.5.1. Gestión del carrito. 18
2.5.2. Ficha emergente del catálogo. 19
2.5.3. Filtros del catálogo. 20
2.5.4. Añadir productos desde el catálogo. 20
2.5.5. Validación de login y registro. 21
2.5.6. Gestión de usuarios y roles. 21
2.5.7. Carrito. 22
2.5.8. Personalizador. 22
2.5.9. Newsletter. 23
2.6. Flujo de datos de una compra. 24
2.7. Flujo de datos de una personalización. 24
2.8. Tecnologías usadas. 25
2.8.1. Metodología BEM. 25
2.8.2. Técnicas de maquetación. 26
2.8.3. Diseño Responsive y Media Queries. 26
Capítulo 3. Sistema de Gestión Empresarial. 37
3.1 Introducción. 37
3.2 Entorno. 37
3.2.1 Preparación del sistema. 37
3.2.2 Instalación de PostgreSQL. 39
3.2.3 Instalación de Odoo. 40
3.2.4 Instalación de Odoo. 40
3.3 Creación de la base de datos y módulos instalados. 41
3.4 Empresa, usuarios y roles. 42
3.4.1 Configuración de la empresa. 42
3.4.2 Importación de datos. 43
3.4.3 Usuarios y permisos. 43
3.5 CRM y clientes. 44

3.6 Productos, componentes e inventario. 46
3.7 Fabricación. 46
3.8 Fabricación bajo pedido (MTO). 47
3.9 Ventas. 48
3.10 Fabricación, inventario y entrega. 49
3.11 Facturación. 50
3.12 Trazabilidad. 52
3.12 Resultados obtenidos. 52
Capítulo 4. Programación de Servicios y Procesos. 53
4.1. Planificación y metodología. 53
4.2. Descripción de la solución. 54
4.2.1 Proceso central. 54
4.2.2 Procesos secundarios. 55
4.2.3 Comunicación entre procesos. 59
4.2.4 Control de ejecución y gestión de errores. 60
4.3 Archivos generados 61
4.4 Resultados Obtenidos 62
Capítulo 5. Gestión de errores. 63
1. Conclusiones del trabajo 64
Capítulo 7. Referencias. 65

Capítulo 1. Resumen del Proyecto.

El proyecto que se va a presentar a continuación consiste en el desarrollo de un sistema completo de gestión para una empresa. Siendo el principal objetivo de este centralizar la administración de datos de una empresa llamada Mercury.

Este trabajo ha sido realizado siguiendo una metodología de trabajo por fases. De esta forma ha sido posible organizar el proceso de desarrollo de forma estructurada y progresiva. La primera de estas fases ha consistido en un proceso de análisis y diseño de la base de datos donde se definieron los productos, modelos y entidades del sistema. Una vez se tomaron estas decisiones, se procedió a crearla, ya que es el núcleo del proyecto y donde se cimientan el resto de fases.

Tras la base de datos, el siguiente paso consistió en la instalación de un entorno Odoo, que representa el sistema central de gestión de la empresa en una máquina virtual de Ubuntu. De esta forma se puede gestionar el sistema empresarial de una forma sencilla y visual.

Tras terminar esta fase, la siguiente consistió en crear la página web de la empresa. Esta web permite a los clientes informarse sobre los distintos productos y modelos disponibles para comprar además de adquirirlos y que esto altere el contenido de la base de datos.

La última fase del proyecto ha consistido en desarrollar un Dashboard web de la base de datos que permite procesar y tratar los datos existentes en la base de datos para conseguir información útil de ellos mediante el análisis de datos. Por otro lado, también se ha creado una aplicación Java que se encarga de procesar los pedidos custom mediante varios procesos simultáneos, con el objetivo de complementar al sistema Odoo.

1.1. Objetivos del proyecto

El proyecto presentado a continuación tiene como objetivo el desarrollo de un sistema de gestión orientado a una empresa dedicada a la venta de zapatillas. El sistema desarrollado tiene como objetivo facilitar la gestión de los pedidos, clientes y productos entre otros mediante las herramientas aprendidas a lo largo de la evaluación.

Este proyecto se enfoca en aplicar de forma práctica los conocimientos adquiridos a lo largo de la evaluación e investigar tecnologías nuevas para la creación del sistema, como por ejemplo Streamlit. Las asignaturas que componen los requisitos del proyecto son Acceso a Datos, Sistemas de Gestión Empresarial, Desarrollo de interfaces, Programación de Servicios y Procesos y Ampliación de Python.

El módulo de Acceso a datos se compone de la creación de una base de datos completa que almacene la información de entre otras cosas pedidos, clientes y productos puestos a la venta. Además de almacenar todos los cambios que ocurran dentro de la misma mediante un sistema de log de Eventos y la separación de responsabilidades mediante un sistema de grupos y usuarios.

Para Ampliación de Python el objetivo consiste en el desarrollo de un dashboard que permite analizar los datos existentes en la base de datos, realizar informes sobre dichos análisis, depurar la base de datos y mostrar estos datos en una página web.

El módulo de Desarrollo de interfaces pide crear una página web en la que los clientes puedan realizar compras y que mediante una conexión con la base de datos se almacene en ella información del pedido y de dicho clientes. Esto es posible mediante dicha conexión y un sistema de inicio de sesión.

En el apartado de Programación de Servicios y Procesos se ha pedido realizar un programa en Java que permita coordinar diversos procesos de forma simultánea para gestionar pedidos personalizados.

En el módulo de Sistemas de Gestión Empresarial se pide la gestion integra de una empresa desde una máquina virtual de ubuntu con el programa de gestión Odoo y su correspondiente base de datos de PostgreSQL.

1.2. Resultados obtenidos

Al final del desarrollo de este proyecto se ha obtenido como resultado final un sistema de gestión de una empresa que consta de varios apartados. Uno de los resultados es una base de datos completamente funcional que almacena entre otras cosas todos los pedidos realizados a la empresa, los trabajadores que se distribuyen entre varias sucursales y las fábricas que ofrecen los productos que vende la empresa.

Otro de los resultados que se han obtenido al final del desarrollo ha sido una página web para el módulo de Lenguaje de Marcas. Esta web se ha dotado de una identidad visual premeditada además de la capacidad del usuario de interactuar con la misma. Mediante el uso de Javascript y los inputs del usuario se ha logrado la creación de pedidos en la base de datos y la modificación de ella.

Complementado a este sistema, también se ha conseguido crear un programa de análisis de datos que permite obtener información fundamental para la futura expansión de la empresa cumpliendo así los requisitos del módulo de Ampliación de Python. El programa en cuestión consiste en un dashboard web desarrollado mediante python usando Streamlit.

Al terminar el apartado de Sistemas de Gestión Empresarial se ha obtenido un entorno Odoo que permite gestionar de forma centralizada los datos de la empresa. Esto no anula a la base de datos si no que actúa como complemento de la misma. Dentro de este sistema se permite gestionar clientes, contactos y trabajadores además de crear pedidos y poder gestionar el stock de los productos y modelos puestos a la venta.

Por último, al terminar con los requisitos de Programación de Servicios y Procesos se ha obtenido un programa desarrollado en Java que permite gestionar los pedidos custom. Además de esto, todos estos resultados se han obtenido siguiendo los principios de la digitalización aprendidos en el módulo que lleva el mismo nombre

Capítulo 2. Desarrollo de Interfaces.

En la parte de Desarrollo de Interfaces se ha realizado una maquetación y estructura que contiene un catálogo de productos estándar y una línea Custom dirigida tanto a clientes independientes como a clubes, grupos y equipos que necesiten diseños personalizados. La actividad de la web abarca ventas, atención comercial, gestión de clientes y seguimiento de pedidos entre muchas otras.

Se ha elegido una identidad de marca propia llamada Mercury, una marca de calzado con el logotipo de estilo tipográfico, mostrado como MERCURY/, aunque también se ha realizado un logotipo de tipo imagen que se muestra en la pestaña de la propia página. Cuenta con un diseño basado en comodidad visual y colección reducida.

Se ha querido mostrar de qué trata la empresa, qué vende, permitir que un cliente compre productos de forma sencilla y lo más destacado, que un cliente pueda iniciar una personalización del producto.

Todo ello con una web multipágina, construida con HTML, CSS y JavaScript sin frameworks, las cuales incluyen página de inicio, catálogo, carritos entre otras.

2.1. Planificación y metodología.

La primera fase del desarrollo web consistió en la maquetación de las distintas páginas que compondrían el sitio. Esta etapa fue fundamental para definir la estructura visual, la distribución de los elementos y la coherencia visual antes de su implementación. Además se distribuyó de forma equitativa para que ninguno de los miembros realizara una carga de trabajo mayor que el resto.

Una vez se decidió la estructura e identidad visual de la página el siguiente paso consistió en crear la web. Se desarrollaron todas las distintas páginas que componen el sitio web, aparte de dotarlas de funcionalidades mediante JavaScript

Por último, la tercera fase consistió en conectar la base de datos con la página web, asegurando una web funcional y enlazarla con el resto del proyecto. Gracias a esto, se consigue que los pedidos realizados en la página web se creen dentro de la base de datos, teniendo en cuenta también si se ha iniciado sesión o no. De esta forma todos estos pedidos se podrán analizar más adelante gracias al dashboard desarrollado en el módulo de Ampliación de Python.

2.2. Identidad visual.

La identidad visual de Mercury es minimalista, es decir, utiliza como color primario el blanco, como color secundario el negro y para mostrar detalles se ha utilizado un color de acento turquesa. Se ha realizado de manera que el protagonismo recaiga sobre el producto. Todo el conjunto está diseñado para ofrecer una interfaz clara, moderna y coherente con el contexto de una empresa de zapatillas.

Para el fondo general de la página se ha optado por utilizar el color blanco, mientras que para determinados cuadros de texto, header o el footer se ha decidido utilizar el negro, lo que genera un contraste visual y guía la visión del usuario a dichas zonas. Además se ha utilizado el color turquesa para acentuar elementos interactuables de la página o información relevante para los usuarios.

La página web se divide en cuatro partes distintas, dedicadas a distintas funciones. La segunda de estas secciones es la página de catálogo. En ella se permite al usuario ver los distintos productos que se hayan a la venta. Al pulsar uno de estos productos, aparece un menú desplegable con información acerca de los productos y permite a los usuarios comprarlos y añadirlos al carrito.

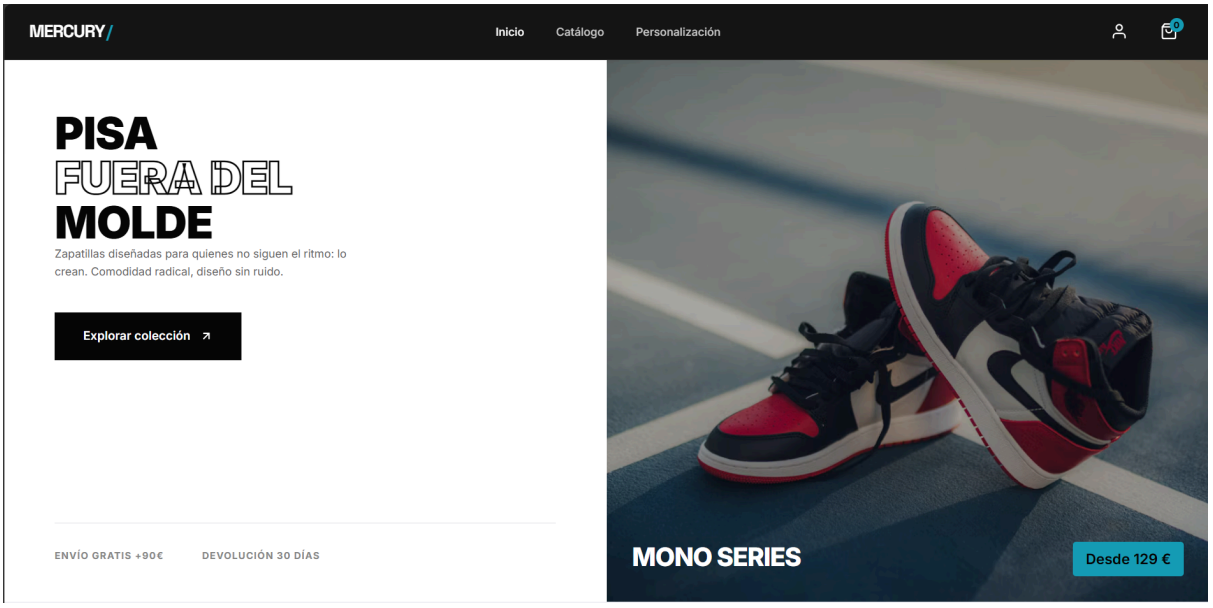
Otra de las secciones principales del modelo consiste en un apartado para personalizar zapatillas. Como su nombre indica, esta sección permite al usuario personalizar algunos de los productos, concretamente zapatillas y calcetines. Además de permitir añadir al carrito dichos productos personalizados para poder comprarlos en un futuro.

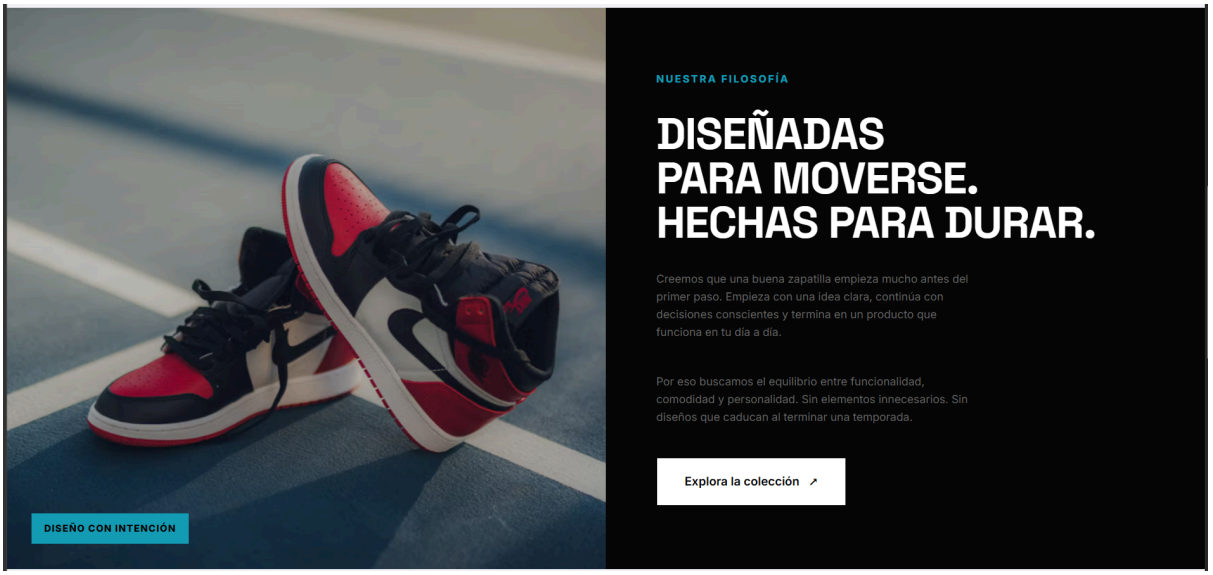
Uno de los apartados de la web más significativos es iniciar sesión. Esta página permite al usuario iniciar sesión o crear un usuario para poder acceder a su pedido. Dicha sección consta de un formulario donde el usuario deberá introducir los datos correspondientes. Una vez se ha creado una cuenta, aparecerá un cliente nuevo en la base de datos.

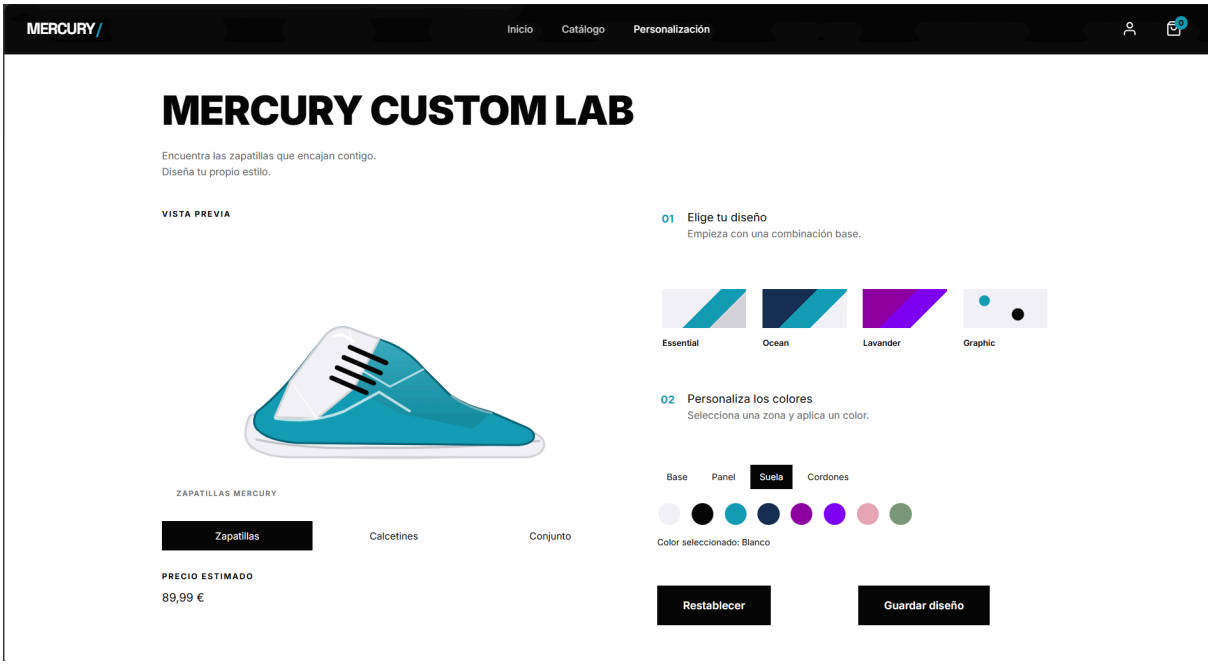
El último apartado a mencionar es el carrito. En dicha página, aparecerán todos los productos que se han ido añadiendo a lo largo de la navegación web y permitirá

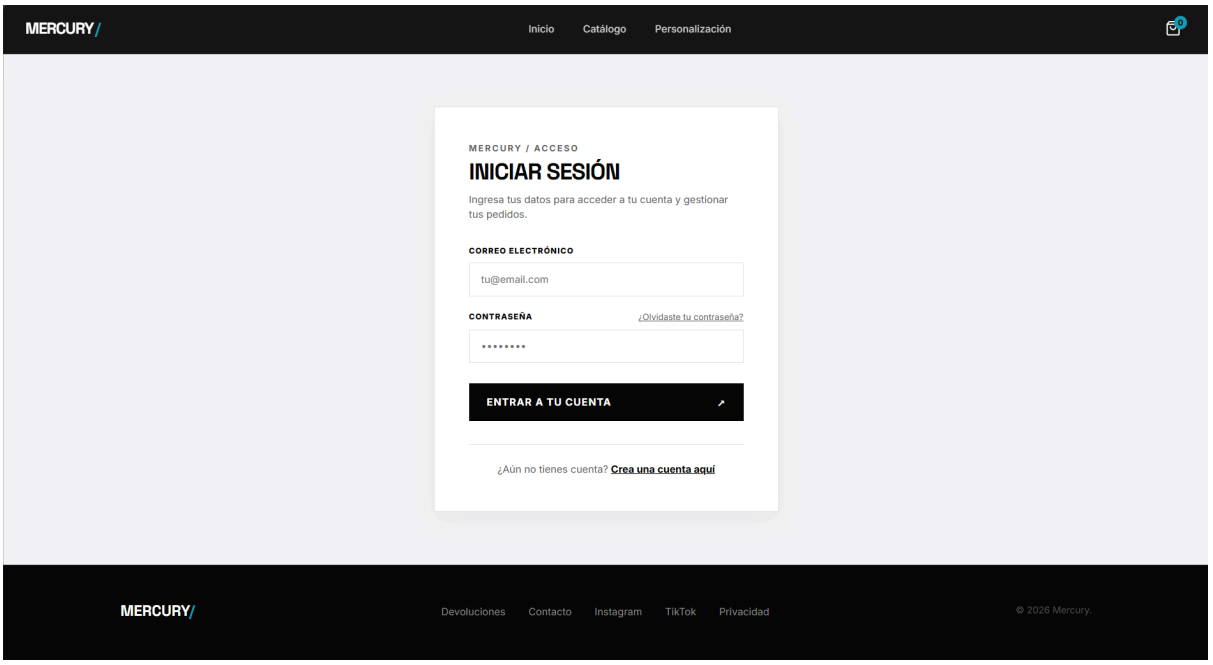
pagar a los usuarios para realizar el pedido y que estos aparezcan en la base de datos.

Gracias a este diseño, se obtiene una navegación clara a lo largo de la página web, mejorando la comprensión de la web y la eficiencia del usuario a lo largo de la compra.









2.2.1. Variables de diseño :root.

En el desarrollo web, las variables de diseño en :root (también llamadas custom properties o propiedades personalizadas de CSS) son valores reutilizables que se declaran en el selector :root para tener un alcance global en todo el documento HTML.

```css
:root {
    --color-black: rgb(8, 8, 8);
    --color-white: rgb(255, 255, 255);
    --color-gray-light: rgb(244, 244, 245);
    --color-gray-border: rgb(228, 228, 231);
    --color-gray-text: rgb(102, 102, 102);
    --color-accent-neon: rgb(23, 159, 181);
    --color-accent-hover: rgb(23, 159, 181);
    --font-display: "Space Grotesk", Arial, sans-serif;
    --font-body: "Inter", Arial, sans-serif;
    --font-main: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
    --font-display: 'Space Grotesk', 'Inter', sans-serif;
    --transition-fast: 0.2s ease;
    --transition-normal: 0.3s ease;
    --container-width: 1400px;
}
```

2.2.2. Tipografía.

- Space Grotesk para titulares y elementos de marca; carácter geométrico y moderno.
- Inter para cuerpo, formularios y navegación; alta legibilidad en pantalla.
- Los titulares utilizan mayúsculas, interletrado y pesos altos para reforzar la jerarquía; los textos descriptivos usan gris y peso normal.
- Se declara una pila de respaldo (Arial, sans-serif o -apple-system, BlinkMacSystemFont, sans-serif) para evitar saltos si falla la carga.

```html
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800;900&family=Space+Grotesk:wght@500;700&display=swap"
```

2.3. Estructura y arquitectura.

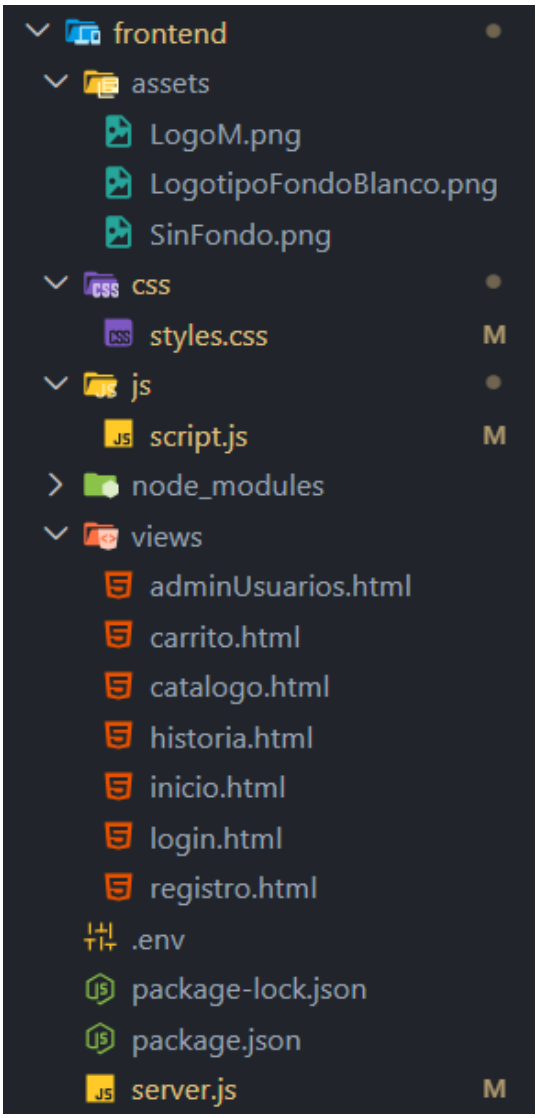
La web indica una estructura de diseño sencillo que prioriza la facilidad de la navegación entre las distintas secciones de la página web y la comodidad del usuario a su vez. Dentro de su estructura principal se incluye estas especificaciones:

- Página de inicio corporativa.
- Catálogo con filtros, ficha emergente de producto y selección de talla.
- Personalizador visual interactivo, llamado Mercury Lab, con vista previa en tiempo real.
- Carrito con control de sesión, resumen de pedido, formulario de envío y pago.
- Registro e inicio de sesión de usuarios con validación en cliente.
- Suscripción de la newsletterf.

Para ello, se ha utilizado como herramienta JavaScript para conseguir las funcionalidades requeridas.
La web sigue una arquitectura cliente-servidor de tres capas lógicas, primero las página HTML, los estilos en CSS y un único fichero JavaScript compartido por todas las páginas. Segundo, la lógica de la aplicación hacia el cliente, el script gestiona el estado del carrito, la sesión visible del usuario, el personalizador, los filtros del catálogo y las validaciones de formularios. Y por último, los servicios en el servidor, gestionado por Api´s, como la autenticación, usuarios, productos y pedidos.
La persistencia en el navegador se limita con localStorage basado en estas dos claves, mercuryCart, es decir el carrito y mercuryUser, el usuario autenticado.

2.3.1. Estructura de ficheros.

La estructura de los ficheros es representada de mejor manera con una imagen.



2.4. Descripción detallada de las páginas.

2.4.1. Inicio (inicio.html)

Propósito: Presentar la marca, su filosofía y llevar al catálogo.

Secciones:

1. Hero. Titular «Pisa fuera del molde» (con parte del texto en contorno), descripción, botón «Explorar colección» y dos mensajes de servicio: envío gratis a partir de 90 € y devolución en 30 días. A la derecha, una imagen de la colección *Mono Series* con rótulo y precio «Desde 129 €».
2. Cifras. Tres indicadores de confianza: 32K pares vendidos, 18 países, valoración 4,9.
3. Filosofía. Imagen y texto «Diseñadas para moverse. Hechas para durar», con enlace al catálogo.
4. Principios. Tres tarjetas numeradas: diseño honesto, comodidad real, menos y mejor.
5. Newsletter. Formulario de correo con mensaje de confirmación y promesa de un 10 % en el primer pedido.

Decisiones: Se evita saturar la portada: pocas secciones, mucho espacio en blanco y llamadas a la acción claras. La sección de cifras y la de principios usan la misma rejilla de tres columnas para mantener el ritmo visual.

2.4.2. Catálogo (catalogo.html)

Propósito: Mostrar los productos con nombre, imagen, precio, categoría y variantes (tallas).

Contenido real del catálogo:

- 23 productos en una única rejilla (#productGrid): 15 zapatillas (de 89 € a 199 €, categorías Lifestyle, Running y Edición limitada) y 8 complementos (calcetines y cordones, de 8 € a 20 €).
- Insignias de producto: NUEVO, LIMITADO y BESTSELLER.
- Cada tarjeta es un article con imagen, insignia, botón de favorito, botón «Añadir al carrito» en superposición, título, categoría y precio. El precio y el identificador del producto viajan en atributos data-* del botón, lo que desacopla el HTML del script.

Filtros. Barra con cinco botones (Todos, Zapatillas, Calcetines, Cordones, Gorras) que filtran por el atributo data-category de las tarjetas.

Ficha emergente (popup). Al pulsar una tarjeta de zapatilla se abre un diálogo modal con imagen ampliada, categoría, insignia, nombre, precio, descripción, selector de tallas (39 a 44), guía de tallas, botón «Añadir a la bolsa» con precio e información de envío y devolución.

Decisiones:

- La ficha de producto se resuelve con una ventana emergente reutilizable, y no con una página por producto, para mantener un flujo ágil y no duplicar plantillas.
- Los datos de la ficha se leen de la propia tarjeta (DOM), por lo que añadir un producto solo exige copiar una tarjeta.
- La talla es obligatoria en zapatillas; en complementos se asigna talla Única.

4.4.3. Personalización - Custom Lab (historia.html)

Propósito: Es el servicio diferencial de la empresa: permitir que el cliente diseñe su producto. La página se titula internamente "Customizer" y en el menú aparece como "Personalización".

Estructura:

1. Hero editorial con el mensaje "No se trata de seguir. Se trata de crear."
2. Cinta animada (marquee) con los valores de marca.
3. Custom Lab: panel de vista previa y panel de controles.

Panel de vista previa. Dos ilustraciones SVG inline (zapatilla de 560×360 y calcetín de 300×360), selector de producto (Zapatillas, Calcetines, Conjunto) y precio estimado.

Panel de controles.

- Paso 01 – Elige tu diseño: cuatro diseños base (Essential, Ocean, Lavander, Graphic).
- Paso 02 – Personaliza los colores: selector de zona (Base, Panel, Suela y Cordones; en calcetines, Base y Puño) y paleta de ocho colores (Blanco, Negro, Turquesa, Azul marino, Morado, Violeta, Rosa, Verde).
- Acciones: «Restablecer» y «Guardar diseño», con mensaje de estado accesible (role=status, aria-live=polite).

Decisión técnica clave. La vista previa utiliza SVG con clases por zona y variables CSS (--shoe-main-color, --shoe-panel-color, --shoe-sole-color, --shoe-laces-color, --sock-main-color, --sock-panel-color). El script solo escribe esas variables en el contenedor; el recoloreado es responsabilidad del CSS. Esto evita manipular cada trazo del SVG desde JavaScript y mantiene separada la lógica de la presentación.

2.4.4.Carrito y finalización de compra (carrito.html)

Propósito: Revisar la selección y completar el pedido.

La página contiene dos estados excluyentes:

1. Acceso requerido (#cartLoginRequired): mensaje "Tu carrito te espera" con botones de iniciar sesión y crear cuenta.

2. Carrito y checkout (#cartCheckout, oculto por defecto): lista de productos, formulario y resumen.

Lista de productos. Cada línea muestra imagen, nombre, talla, precio total de la línea, controles de cantidad (−/+) y botón "Eliminar producto". Si no hay productos, aparece un estado vacío con enlace al catálogo.

Formulario en tres bloques:

1. Datos del destinatario: nombre completo, correo (precargado desde la sesión) y teléfono.
2. Dirección de envío: dirección, número/piso/puerta, código postal, ciudad, provincia e indicaciones opcionales.
3. Método de pago: tarjeta bancaria (titular, número, caducidad, CVV) con aviso de que los datos de la tarjeta no deben almacenarse en el navegador.

Incluye casilla obligatoria de aceptación de condiciones y botón «Confirmar pedido».

Resumen lateral (aside). Productos, envío, total e información de compra. El envío se fija en 0 € (SHIPPING_PRICE = 0).

2.4.5. Inicio de sesión (login.html) y registro (registro.html)

Ambas páginas comparten la tarjeta de autenticación (auth__card) y la validación en cliente.

- Login: correo y contraseña (mínimo 8 caracteres) y enlace al registro.
- Registro: nombre, apellidos, correo, contraseña (mínimo 8 caracteres) y aceptación de términos y política de privacidad.
- Ambos formularios usan novalidate para controlar los mensajes desde JavaScript y mostrar errores contextuales bajo cada campo.

2.4.6. Gestión de usuarios (adminUsuarios.html)

Propósito: Panel de administración para consultar las cuentas y cambiar roles. Se aloja en una página propia y su cabecera solo ofrece volver a la tienda.

- Buscador por nombre o correo con filtrado en vivo.
- Contador de usuarios.
- Tabla con usuario, correo, rol actual y selector para modificar el rol.
- Mensajes de éxito o error en una región aria-live.
- Roles disponibles: cliente, administrador, almacén, comercial y encargado.

2.5. JavaScript.

Todo el comportamiento se concentra en script.js, cargado al final del body de cada página. Como el mismo fichero se carga en todas las páginas, cada módulo

comprueba primero que existen en el DOM los elementos que necesita (por ejemplo if (!cartItems) return;) y, si no, no hace nada. Así una única base de código sirve a siete páginas sin errores en consola.

La organización del código se basa en la región en la que queremos actuar y las responsabilidades que le damos con el propio JavaScript. A la base de datos la responsabilidad son las constantes y la petición inicial de los productos; a la gestión del carrito se le asocia la responsabilidad de la lectura, escritura, contador e indicador de añadido; al pop-up del catálogo se le indica la responsabilidad de la ficha emergente y selección de talla; a los filtros del catálogo se le asocia el filtrado con transición; a la acción de añadir al carrito desde el catálogo se le indica la responsabilidad de la construcción del objeto producto; al login y registro se le asocia la validación y llamadas de a la Api; a la gestión de usuarios se le responsabiliza del listado, búsqueda y cambio de rol; a la página del carrito se le indica el render, cantidades y checkout; al personalizador se le responsabiliza del estado, colores, precio y guardado.

Las constantes y utilidades globales del servidor se resumen en estas:

- API_URL = '/api': prefijo único de todas las rutas del servidor, lo que permite cambiar de entorno en un solo punto.
- CART_KEY = 'mercuryCart' y USER_KEY = 'mercuryUser': claves de localStorage.
- SHIPPING_PRICE = 0: coste de envío.
- currency: instancia de Intl.NumberFormat('es-ES', { style: 'currency', currency: 'EUR' }), que da formato monetario español sin concatenar cadenas a mano.

2.5.1. Gestión del carrito.

El carrito es un array de objetos guardado como JSON en localStorage. La funciones principales que destacan son las siguientes:

- getCart(): lee el carrito dentro de un try/catch; si el JSON está corrupto devuelve un array vacío en lugar de romper la página.
- saveCart(cart): guarda, actualiza la insignia y emite el evento personalizado cartUpdated, de modo que cualquier parte de la interfaz interesada pueda reaccionar (la página del carrito lo escucha para repintarse).
- getCartCount(): suma las cantidades con reduce.
- updateCartBadge(): actualiza todos los elementos #cartCount.
- addProductToCart(product): si ya existe una línea con el mismo id y la misma talla, y ninguna es personalizada, incrementa su cantidad; en otro caso añade una línea nueva con quantity: 1. Los productos personalizados nunca se fusionan, porque dos diseños distintos son productos distintos.

- getCurrentUser(): recupera el usuario de sesión guardado tras el login.

```
    /* Recuperar productos */
function getCart() {
    try {
        return JSON.parse(localStorage.getItem(CART_KEY)) || [];
    } catch (error) {
        console.error('Error al recuperar el carrito:', error);
        return [];
    }
}

    /* Guardar productos */
function saveCart(cart) {
    localStorage.setItem(CART_KEY, JSON.stringify(cart));

    updateCartBadge();

    document.dispatchEvent(new CustomEvent('cartUpdated'));
}

    /* Número total de unidades */
function getCartCount() {
    return getCart().reduce((total, product) => {
        return total + (Number(product.quantity) || 1);
    }, 0);
}

    /* Actualizar contador en todas las páginas */
function updateCartBadge() {
    const badges = document.querySelectorAll('#cartCount');

    badges.forEach(badge => {
        badge.textContent = getCartCount();
    });
}
```

2.5.2. Ficha emergente del catálogo.

Se usa delegación de eventos: un único listener sobre #productGrid atiende los clics de todas las tarjetas, incluidas las que se añadan en el futuro. Si el clic es sobre el botón «Añadir al carrito», no se abre la ficha. Si el clic cae en una tarjeta de zapatilla, el script copia imagen, nombre, categoría, insignia y precio (leído de data-price) al popup. Se reinicia la talla seleccionada y se abre el popup añadiendo popup--active, actualizando aria-hidden y bloqueando el scroll del cuerpo con la clase product-popup-open. El usuario elige talla (clase popup__size--active). Al pulsar «Añadir a la bolsa»: si no hay talla, aviso; si la hay, se guarda en el botón original (dataset.size) y se simula su clic, reutilizando así la misma lógica de añadido en un solo lugar.
Cierre del popup: botón ×, clic en el fondo oscuro y tecla Escape.

2.5.3. Filtros del catálogo.

Cada botón lleva data-filter; cada tarjeta, data-category. Al filtrar, las coincidentes se muestran y las demás se desvanecen y se ocultan tras 200 ms. El botón activo recibe el modificador catalog__filter-btn--active. El retardo de 200 ms permite que la animación de salida termine antes de retirar el elemento del flujo.

```
/* FILTROS DEL CATÁLOGO */

    const filterBtns = document.querySelectorAll('.catalog__filter-btn');

    const productCards = document.querySelectorAll(
        '.catalog__grid .card, #productGrid .card'
    );

    filterBtns.forEach(btn => {
        btn.addEventListener('click', () => {
            filterBtns.forEach(button => {
                button.classList.remove('catalog__filter-btn--active');
            });

            btn.classList.add('catalog__filter-btn--active');

            const filterValue = btn.getAttribute('data-filter');

            productCards.forEach(card => {
                const category = card.getAttribute('data-category');

                if (filterValue === 'all' || category === filterValue) {
                    card.style.display = 'flex';

                    setTimeout(() => {
                        card.style.opacity = '1';
                        card.style.transform = 'scale(1)';
                    }, 50);
                } else {
                    card.style.opacity = '0';
                    card.style.transform = 'scale(0.95)';

                    setTimeout(() => {
                        card.style.display = 'none';
                    }, 200);
                }
            });
        });
```

2.5.4. Añadir productos desde el catálogo.

El script construye el producto leyendo data-id y data-price del botón y el nombre y la imagen de la tarjeta. Si es zapatilla y no hay talla, muestra un aviso y no añade nada. Tras añadir, el texto del botón cambia durante 1,2 s a “¡Añadido!” con el color de acento, como confirmación visual inmediata, y luego se restaura.

2.5.5. Validación de login y registro.

Se valida con correo: expresión regular ^[^\s@]+@[^\s@]+\.[^\s@]+$., contraseña: mínimo 8 caracteres, nombre y apellidos: no vacíos (tras trim)y términos: casilla obligatoria.
Mensajes: Se muestran con las clases auth__form-input--error y auth__form-error--active, y desaparecen cuando el campo vuelve a ser válido. Las funciones showError y clearError evitan repetir código.

Envío: El botón se deshabilita y cambia su texto mientras dura la petición, para evitar dobles envíos; se restaura en finally. Si el servidor rechaza la petición se muestra su mensaje; si no hay conexión (Failed to fetch) se informa de que no se pudo conectar. Tras un login correcto se guarda el usuario devuelto y se redirige a Inicio; tras un registro correcto, a Login.

```
document.addEventListener('DOMContentLoaded', () => {
    const isValidEmail = email => {
        const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        return re.test(String(email).toLowerCase());
    };

    const showError = (input, error) => {
        if (input) input.classList.add('auth__form-input--error');
        if (error) error.classList.add('auth__form-error--active');
    };

    const clearError = (input, error) => {
        if (input) input.classList.remove('auth__form-input--error');
        if (error) error.classList.remove('auth__form-error--active');
    };
```

2.5.6. Gestión de usuarios y roles.

Para la gestión de usuarios y roles es necesario estos métodos:

- loadUsers() solicita GET /api/users con credenciales; si el servidor responde con error muestra «No tienes permiso para consultar los usuarios».
- renderUsers(search) filtra por nombre completo o correo (sin distinguir mayúsculas), actualiza el contador y reconstruye la tabla. Las celdas se crean con createElement y textContent, no con innerHTML, lo que evita inyección de HTML en los datos de usuario.
- Cada fila lleva un select con los cinco roles y aria-label con el nombre del usuario.

- updateUserRole() envía PATCH /api/users/:id/role con { rol }, deshabilita el selector durante la petición, actualiza el estado local y muestra confirmación; si falla, informa del error y recarga la lista para que la interfaz refleje el estado real del servidor.

2.5.7. Carrito.

En la página del carrito se utilizan estos métodos para que sea efectivo tanto la gestión del propio como para que solo puedas utilizarlo con la sesión iniciada:

- checkSession() decide qué estado mostrar (acceso requerido o checkout) según exista usuario, y precarga el correo del destinatario.
- renderCart() reconstruye la lista con createElement y calcula número de productos, subtotal, envío y total. Si una imagen falla, se sustituye por el logotipo (onerror). También gestiona los estados vacío y con productos (mostrar u ocultar lista, formulario y botón de confirmar).
- Delegación de eventos sobre #cartItems con atributos data-action (increase, decrease, remove) y data-index. Reducir la cantidad de una línea de 1 la elimina.
- Confirmación del pedido. Se comprueba sesión y carrito no vacío, se valida el formulario con reportValidity(), se monta el objeto destinatario y se envía api/pedidos. En éxito se vacía el carrito, se reinicia el formulario y se muestra el número de pedido y el total; en error, el mensaje en rojo y el botón se reactiva.

2.5.8. Personalizador.

En el personalizador es necesario la utilización de estos métodos:

- getActiveColors() y getAvailableZones() devuelven la parte del estado y las zonas válidas según el producto (los calcetines solo tienen Base y Puño).
- renderProduct() muestra u oculta cada SVG, actualiza el pie de la ilustración, los botones activos y llama a las demás funciones de pintado.
- renderPalette() genera los ocho botones de color con aria-label, title y aria-pressed.
- renderColors() vuelca el estado a las variables CSS del contenedor de la vista previa.
- renderPrice() y renderSummary() actualizan el precio y el texto resumen (por ejemplo, «Zapatillas Essential, base blanco. Precio estimado: 89,99 €»).
- setDesign() aplica un preajuste de colores a todas las zonas.
- Restablecer copia en profundidad el estado inicial (JSON.parse(JSON.stringify(...))) para no mutar la constante original.
- Guardar crea un producto con id: 'custom-' + Date.now(), talla Personalizado y el objeto customization, y lo añade al carrito.

-

```
/* PERSONALIZADOR MERCURY */

document.addEventListener('DOMContentLoaded', () => {
    const stage = document.getElementById('customizerStage');

    if (!stage) return;
    const shoePreview = document.getElementById('shoePreview');
    const sockPreview = document.getElementById('sockPreview');

    const stageCaption = document.getElementById('stageCaption');

    const priceValue = document.getElementById('priceValue');

    const colorPalette = document.getElementById('colorPalette');

    const selectedColorLabel =
        document.getElementById('selectedColorLabel');

    const designSummary = document.getElementById('designSummary');

    const feedback = document.getElementById('feedback');

    const sizeOptions = document.getElementById('sizeOptions');

    const selectedSizeLabel = document.getElementById('selectedSizeLabel');

    const colors = [
        { name: 'Blanco', value: '#f4f4f5', className: 'white' },
        { name: 'Negro', value: '#080808', className: 'black' },
        { name: 'Turquesa', value: '#179fb5', className: 'blue' },
        { name: 'Azul marino', value: '#142e50', className: 'navy' },
        { name: 'Morado', value: '#8c04a0', className: 'purple' },
        { name: 'Violeta', value: '#7e04f0', className: 'violet' },
        { name: 'Rosa', value: '#e7a5b5', className: 'pink' },
        { name: 'Verde', value: '#78977b', className: 'green' }
    ];
```

2.5.9. Newsletter.

El formulario evita el envío real, muestra durante 4 s un mensaje de agradecimiento mediante la clase newsletter__feedback--success y reinicia el campo. Es un prototipo: no envía el correo a ningún servicio.

```
/* NEWSLETTER */

document.addEventListener('DOMContentLoaded', () => {
    const newsletterForm =
        document.getElementById('newsletterForm');

    const newsletterFeedback =
        document.getElementById('newsletterFeedback');

    if (newsletterForm) {
        newsletterForm.addEventListener('submit', event => {

            event.preventDefault();

            if (newsletterFeedback) {
                newsletterFeedback.classList.add(
                    'newsletter__feedback--success'
                );
            }

            newsletterForm.reset();

            setTimeout(() => {
                if (newsletterFeedback) {
                    newsletterFeedback.classList.remove(
                        'newsletter__feedback--success'
                    );
                }
            }, 4000);
        });
    }
```

2.6. Flujo de datos de una compra.

1. El usuario navega por el catálogo y abre la ficha de un producto.
2. Elige talla y añade el producto; el script lo guarda en localStorage y actualiza el contador del carrito en todas las páginas.
3. En carrito.html, si no hay sesión se muestra una pantalla de acceso; si la hay, se muestra el carrito y el formulario de envío.
4. Al confirmar, el cliente envía a la api/pedidos con el usuario, el destinatario y las líneas del carrito.
5. El servidor responde con el identificador y el total del pedido, así vaciándose el carrito.

2.7. Flujo de datos de una personalización.

1. En la página de personalización el usuario elige el producto (zapatillas, calcetines o conjunto), diseño base y colores por zona.
2. El script mantiene un objeto de estado y refleja los colores en la vista previa SVG mediante variables CSS.
3. Al pulsar «Guardar diseño», se crea una línea de carrito con customization (producto, diseño y colores) y talla personalizada.
4. Esa línea viaja con el pedido al servidor.

2.8. Tecnologías usadas.

HTML, se ha utilizado para la estructura de todas las páginas, header, nav, main, section, article, footer, form, entre ellas.
CSS, se ha utilizado para la presentación, variables CSS, flexbox, grid, animaciones. Todo ellos bajo la nomenclatura BEM para una buena estructura de clases.
JavaScript, utilizado para la interactividad de las páginas, fetch, async/await, localStorage, eventos personalizados.
SVG, para una vista previa de los calcetines y zapatillas en la sección custom, permite colorear por zonas con las variables recogidas en el CSS.
Google Fonts, una herramienta utilizada para las tipografías de cuerpo y titulares.
Unsplash, usada para las imágenes de productos, con ello creamos material ficticio.

2.8.1. Metodología BEM.

La hoja de estilos usa la nomenclatura BEM (Bloque__Elemento--Modificador), por ejemplo card__overlay-btn--action, catalog__filter-btn--active, popup--active, customizer__swatch--selected, auth__form-input--error.

Esto conlleva ventajas a la hora de una estructura limpia sin anidar selectores o nombres, especificidad evitando los duplicados e indicando los estados expresados con modificadores, que el JavaScript añade y quita con classList en lugar de escribir estilos en línea.

Componentes reutilizados entre páginas:

- Cabecera: .header, .header__nav, .header__menu
- Botones: .common__btn-black, .common__btn-white
- Títulos y descripciones: .common__title, .common__title-secundary, .common__description, .common__tag
- Tarjeta de producto: .card, .card__image-wrapper, .card__details
- Ventana emergente: .popup, .popup__content
- Tarjeta de autenticación: .auth__card, .auth__form
- Pie de página: .footer, .footer__content
- Insignia de carrito: .header__cart-badge#cartCount

2.8.2. Técnicas de maquetación.

- Flexbox para barras de navegación, alineaciones, filas de acciones y componentes pequeños.
- CSS Grid para estructuras bidimensionales: rejilla de productos, tres columnas de cifras y principios, hero a dos columnas (1fr 1fr), personalizador (minmax(0,1.08fr) minmax(360px,.92fr)), carrito (minmax(0,1.6fr) minmax(280px,0.8fr)), ficha de producto (1.1fr 1fr), selector de tallas (repeat(6, minmax(0,1fr))) y líneas del carrito (125px minmax(0,1fr) auto).
- Posicionamiento sticky en la cabecera, de modo que la navegación siempre está disponible.
- backdrop-filter con fondo semitransparente en la cabecera.
- Animación @keyframes scroll en la cinta de texto (marquee) de la página de personalización, con un bucle lineal infinito de 25 s.

2.8.3. Diseño Responsive y Media Queries.

Para garantizar que la página web de MERCURY se visualice correctamente en diferentes dispositivos, se ha implementado un sistema de diseño responsive mediante el uso de media queries en CSS.

El diseño responsive permite adaptar automáticamente la estructura, distribución, dimensiones, espaciados y elementos visuales de una página web en función de las características de la pantalla desde la que se accede. De esta manera, se consigue que la experiencia de navegación sea adecuada tanto en ordenadores de sobremesa como en portátiles, tabletas y teléfonos móviles.

Para conseguirlo, se han establecido diferentes puntos de ruptura (breakpoints) utilizando la propiedad max-width. Esta propiedad permite aplicar determinadas reglas CSS cuando el ancho de la ventana del navegador es igual o inferior al valor indicado.

Es importante destacar que las media queries no sustituyen los estilos CSS originales, sino que los complementan o modifican cuando se cumplen las condiciones establecidas.

En el proyecto se han definido los siguientes puntos de ruptura:

- 1024 px: Tabletas y portátiles pequeños.
- 900 px: Pantallas que necesitan pasar de dos columnas a una.
- 768 px: Móviles grandes y tabletas pequeñas.

- 480 px: Teléfonos móviles.
- 360 px: Teléfonos de dimensiones reducidas
- hover: none: Dispositivos que no disponen de cursor convencional.
- prefers-reduced-motion: Usuarios que prefieren reducir las animaciones

Adaptación a tabletas y portátiles pequeños (1024 px)

.customizer__layout: La propiedad grid-template-columns: minmax(0, 1fr) establece una columna que ocupa el espacio disponible. Con ello, se evita mantener una distribución horizontal que podría resultar demasiado estrecha en una tableta.

.customizer__stage: Esto permite mantener un espacio adecuado para mostrar la previsualización del producto, evitando que la zona donde aparece la zapatilla personalizada quede demasiado reducida.

.cart__layout: El contenido principal del carrito se reorganiza en una única columna mediante grid-template-columns. En pantallas grandes, la distribución puede mostrar los productos y el resumen del pedido en columnas independientes. Sin embargo, en pantallas más pequeñas resulta conveniente colocarlos verticalmente para mejorar la lectura.

.cart__summary: Se modifica position a static. Esta propiedad elimina el posicionamiento especial que pudiera mantener el resumen del pedido en una posición fija o relativa durante el desplazamiento por la página.

.catalog__grid: Esta configuración establece dos columnas de igual tamaño, permitiendo mostrar los productos en una cuadrícula equilibrada. Se utiliza minmax(0, 1fr) para permitir que las columnas puedan reducirse sin que el contenido provoque desbordamientos horizontales.

.popup__info: Con esta modificación se reduce el espacio horizontal que ocupa la información del producto, manteniendo una separación adecuada respecto a los bordes de la ventana emergente. El objetivo es conseguir que el nombre, la descripción, el precio y el resto de información del producto puedan visualizarse correctamente.

Adaptación de estructuras de dos columnas a una columna (900 px)

.hero: De esta forma, el contenido textual y el contenido visual dejan de aparecer uno al lado del otro y pasan a organizarse verticalmente. También se establece min-height: 0 para eliminar una altura mínima innecesaria y permitir que el componente se adapte al contenido disponible.

.hero__content: Se ajusta el relleno interno a 48 píxeles en la parte superior, 24 píxeles en los laterales y 32 píxeles en la parte inferior. También se establece una separación de 40 píxeles entre los elementos internos mediante gap.

.hero__visual: Esta modificación garantiza que la imagen principal del producto conserve un tamaño adecuado después de pasar a una distribución vertical.

.hero__footer-info: Se modifica la separación entre los elementos y se activa flex-wrap: wrap. Esta propiedad permite que los elementos pasen automáticamente a una nueva línea cuando no existe espacio horizontal suficiente. Así se evita que la información secundaria del apartado principal quede comprimida o sobresalga de su contenedor.

.story__feature: De esta manera, la imagen y el contenido textual de la sección se muestran uno debajo del otro en lugar de permanecer distribuidos horizontalmente.

.story__feature-visual: Se establece una altura mínima de 320 píxeles para mantener una zona visual proporcionada.

.story__feature-content: Se modifica el relleno interno a 48 píxeles verticales y 24 píxeles horizontales. Esto permite adaptar el texto a la nueva distribución y conservar una separación adecuada respecto a los bordes.

.story__hero: Se transforma la estructura de dos columnas en una sola columna mediante grid-template-columns. Esta modificación permite que la imagen y el contenido editorial se organicen verticalmente. También se establece min-height: 0 para evitar que la sección mantenga una altura mínima excesiva.

.story__hero-image: Con ello se consigue que la imagen mantenga unas dimensiones controladas y proporcionales en pantallas de menor tamaño.

.story__hero-content: Se ajusta el relleno interno a 48 píxeles verticales y 24 horizontales para adaptar el contenido a la nueva estructura.

.popup__content: Se reorganiza la ventana emergente del producto para que su imagen y su información aparezcan en una única columna. Se incluyen las dos propiedades de altura para ofrecer compatibilidad con navegadores que no admiten dvh, utilizando esta última como alternativa más precisa cuando está disponible.

.popup__visual: Se establece una altura de 280 píxeles para la zona visual del producto. Esto permite que la imagen conserve suficiente protagonismo sin ocupar toda la pantalla.

.popup__info: Se modifica justify-content a flex-start para colocar la información al principio del espacio disponible. También se reduce el relleno interno a 32 píxeles en la parte superior, 24 en los laterales y 28 en la parte inferior.

.popup__header: Se establece un margen inferior de 24 píxeles para separar la cabecera de la información que aparece a continuación.

.popup__description: Se aplica también un margen inferior de 24 píxeles para mejorar la separación entre la descripción y los siguientes elementos.

.popup__close: Se modifica la posición del botón de cierre para adaptarlo a la nueva distribución. Se establecen 12 píxeles de separación respecto a los bordes superior y derecho y unas dimensiones de 42 × 42 píxeles. Esto facilita su localización y utilización en pantallas táctiles.

Adaptación a móviles grandes (768 px)

.header__nav: Se modifica la altura a automática mediante height: auto y se permite que sus elementos puedan distribuirse en varias líneas utilizando flex-wrap: wrap. También se ajusta el relleno interno y se establece una separación vertical de 10 píxeles entre las filas. Esto permite que la cabecera se reorganice sin necesidad de modificar la estructura HTML original.

.header__menu: Se utiliza order: 3 para colocar el menú de navegación después de los demás elementos de la cabecera. Se establece width: 100% para que ocupe toda la anchura disponible y justify-content: space-between para distribuir los enlaces de manera equilibrada. Además, se añade un borde superior mediante border-top, que sirve como separador visual entre el menú y el resto de elementos de la cabecera.

.header__actions: Se reduce la separación entre los botones y elementos de acción de la cabecera a 8 píxeles. De esta forma se consigue aprovechar mejor el espacio horizontal disponible.

.container__numbers: Se establece un relleno horizontal de 20 píxeles para evitar que el contenido numérico quede demasiado cerca de los bordes de la pantalla.

.story__background: Se modifica el relleno vertical a 48 píxeles para reducir el espacio ocupado por la sección y mantener una separación equilibrada.

.story__numbers-grid y .story__values-grid: Ambas cuadrículas pasan a tener una única columna mediante grid-template-columns. También se establece una separación de 16 píxeles entre los elementos. Esta modificación permite que los

números y valores de la marca se muestren verticalmente, facilitando su lectura en dispositivos móviles.

.story__number y .story__value: Se establece min-height: 0 para eliminar alturas mínimas que podrían provocar espacios vacíos innecesarios.

.story__values-header: Se establece un margen inferior de 32 píxeles para mantener una separación suficiente entre el encabezado de la sección y su contenido.

.common__description: Se elimina la limitación de anchura mediante max-width: 100% y se ajusta el margen inferior a 28 píxeles. Esto permite que las descripciones aprovechen el espacio horizontal disponible.

.catalog: Se reduce el relleno vertical a 32 píxeles para adaptar la altura de la sección a dispositivos móviles.

.catalog__header: Se ajustan los márgenes y el relleno interno, estableciendo 15 píxeles en vertical y 16 en horizontal. Esto permite que el encabezado del catálogo ocupe menos espacio.

.catalog__filters: Se modifica el margen inferior a 28 píxeles y se ajusta el relleno horizontal a 16 píxeles. También se reduce la separación entre los filtros a 8 píxeles para facilitar su distribución.

.catalog__filter-btn: Se reducen las dimensiones internas de los botones de filtrado y el tamaño de su texto. Se establece un relleno de 9 píxeles en vertical y 16 en horizontal, junto con un tamaño de fuente de 13 píxeles. De esta manera, los filtros ocupan menos espacio sin perder legibilidad.

.catalog__grid: Se ajusta el relleno horizontal a 16 píxeles y se modifica la separación entre los productos. Se establecen 20 píxeles de separación vertical y 16 píxeles de separación horizontal.

.card__overlay-btn: En dispositivos móviles, los botones de las tarjetas deben estar disponibles sin depender del efecto hover. Por este motivo, se elimina la transformación vertical mediante transform: translateY(0). También se ajustan el relleno y el tamaño de fuente. Esta modificación es importante porque los dispositivos táctiles no utilizan el ratón y, por tanto, no pueden depender de los efectos hover de la misma manera que un ordenador.

.customizer: Se establece una anchura del 100 % y se reduce el relleno interno a 40 píxeles en la parte superior, 20 en los laterales y 56 en la parte inferior. Esto permite aprovechar el espacio disponible sin generar márgenes excesivos.

.customizer__visual, .customizer__shoe-svg y .customizer__sock-svg: Se establece una altura de 260 píxeles para los elementos visuales del personalizador. Con ello se consigue que la representación de la zapatilla y los elementos gráficos mantengan unas dimensiones adecuadas.

.customizer__stage Se establece una altura mínima de 300 píxeles y un relleno interno de 16 píxeles. Esto garantiza que la zona de previsualización disponga de espacio suficiente.

.customizer__stage--set .customizer__visual, .customizer__stage--set .customizer__shoe-svg y .customizer__stage--set .customizer__sock-svg: Cuando el personalizador utiliza el estado o modificador --set, se reduce la altura de los elementos visuales a 230 píxeles. Esto permite ajustar la representación del producto cuando se encuentra en esa configuración específica.

.cart: Se modifica el relleno vertical a 48 píxeles y se establece min-height: 0. Esto evita que la sección del carrito mantenga una altura mínima innecesaria en pantallas pequeñas.

.cart__form-section: Se establece un relleno interno de 22 píxeles para reducir el espacio ocupado por los formularios.

.cart__submit: Se ajusta el relleno a 20 píxeles para adaptar el botón o sección de confirmación del pedido.

.cart__summary: Se reduce el relleno interno a 22 píxeles para que el resumen de compra se adapte mejor a la anchura disponible.

.users: Se establece un relleno de 40 píxeles en vertical y 20 en horizontal. Esto permite adaptar el espacio general de la sección de administración.

.users__toolbar: Se cambia la dirección de los elementos flexibles a columna mediante flex-direction: column. También se utiliza align-items: stretch para que los elementos puedan ocupar toda la anchura disponible. Se establece una separación de 12 píxeles entre ellos. De esta manera, los controles de búsqueda y las herramientas de administración se organizan verticalmente.

.users__search: Se elimina la limitación de anchura máxima mediante max-width: none. Así, el campo de búsqueda puede ocupar todo el espacio disponible.

.users__table: Se establece min-width: 640px para mantener una anchura mínima que permita conservar la legibilidad de las columnas de la tabla. Esto significa que, si la pantalla es más estrecha, será necesario disponer de desplazamiento horizontal en el contenedor de la tabla para consultar toda la información sin comprimir excesivamente las columnas.

Adaptación a teléfonos móviles (480 px)

.container: Se establece un relleno horizontal de 16 píxeles. Esto evita que los elementos del sitio web queden pegados a los bordes de la pantalla.

.common__btn-black y .common__btn-white: Se reducen las dimensiones internas de los botones y el tamaño de su texto. Se establece un relleno de 15 píxeles en vertical y 24 en horizontal, con un tamaño de fuente de 14 píxeles. Esto permite mantener botones cómodos de utilizar sin que ocupen demasiado espacio.

.hero__content: Se reduce el relleno interno a 36 píxeles en la parte superior, 16 en los laterales y 24 en la parte inferior. Esto adapta el contenido principal a las dimensiones reducidas de los teléfonos.

.hero__visual: Se establece una altura mínima de 300 píxeles para ajustar la imagen principal.

.hero__overlay: Se ajusta la posición del contenido superpuesto a la imagen mediante márgenes de 16 píxeles respecto a sus bordes.Esto permite mantener una distancia visual adecuada.

.hero__product-title: Se reduce el tamaño del título del producto a 24 píxeles para evitar que ocupe demasiado espacio horizontal.

.hero__price-tag: Se ajustan el tamaño de fuente y el relleno interno de la etiqueta del precio para mantener una presentación proporcionada.

.story__feature-content: Se reduce el relleno interno a 40 píxeles en vertical y 16 en horizontal.

.story__feature-image: Se modifica la posición de la imagen mediante left: 16px y bottom: 16px. De esta forma, se mantiene una separación respecto a los bordes del contenedor.

.story__hero-content: Se ajusta el relleno interno a 40 píxeles verticales y 16 horizontales.

.story__value: Se establece un relleno de 24 píxeles para adaptar los bloques informativos a la pantalla.

.catalog__grid: Se reduce el relleno horizontal a 12 píxeles y se establece una separación de 20 píxeles entre filas y 12 entre columnas. Se mantiene la distribución de dos columnas definida en el punto de ruptura anterior, pero se reduce el espacio ocupado por cada elemento. Esto permite mostrar varios productos sin que las tarjetas resulten demasiado estrechas.

.catalog__header y .catalog__filters: Se ajusta el relleno horizontal a 12 píxeles para mantener la alineación con el resto del catálogo.

.card__details: Se modifica flex-direction a column. Esto permite que el nombre y la categoría del producto se organicen verticalmente en lugar de permanecer en una misma línea. También se establece una separación de 4 píxeles entre ambos elementos.

.card__title: Se reduce el tamaño del título del producto a 15 píxeles.

.card__category: Se establece un tamaño de fuente de 12 píxeles para la categoría.

.card__price: Se ajusta el tamaño del precio a 15 píxeles para mantener una jerarquía visual adecuada.

.card__badge: Se modifica la posición de la etiqueta del producto, estableciendo 8 píxeles respecto a los bordes superior e izquierdo. También se reduce su tamaño de fuente a 10 píxeles y se ajusta su relleno interno.

.card__favorite-btn: Se modifica la posición del botón de favoritos y se reducen sus dimensiones a 32 × 32 píxeles.

.card__overlay-btn: Se reduce el relleno interno a 10 píxeles y el tamaño de fuente a 12 píxeles.Esto permite que el botón de añadir al carrito se adapte al espacio disponible en las tarjetas.

.popup: Se establece padding: 0 para eliminar los márgenes internos de la ventana emergente. Esto permite aprovechar toda la superficie disponible del teléfono.

.popup__content: Se establece una altura de 100vh y posteriormente 100dvh. La segunda propiedad permite adaptar la ventana emergente a la altura dinámica del navegador móvil. Con ello se consigue que la ficha de producto pueda ocupar toda la pantalla.

.popup__visual: Se reduce la altura de la zona visual a 220 píxeles para reservar espacio suficiente para la información del producto.

.popup__info: Se establece un relleno de 24 píxeles en la parte superior, 16 en los laterales y 24 en la parte inferior.

.popup__title: Se utiliza clamp(34px, 11vw, 48px) para establecer un tamaño de fuente adaptable. Esta función permite que el tamaño del título cambie según la anchura de la ventana, respetando un mínimo de 34 píxeles y un máximo de 48 píxeles. También se reduce el espaciado entre letras mediante letter-spacing: -2px.

.popup__description: Se establece un tamaño de fuente de 14 píxeles para mejorar la adaptación del texto.

.popup__shipping: Se modifica la dirección de los elementos flexibles a columna y se establece una separación de 6 píxeles. Esto permite organizar verticalmente la información de envío.

.customizer: Se reduce el relleno interno a 32 píxeles en la parte superior, 16 en los laterales y 48 en la parte inferior.

.customizer__designs: Se establece una cuadrícula de dos columnas mediante repeat(2, minmax(0, 1fr)). Esto permite mostrar las diferentes opciones de diseño de forma ordenada.

.customizer__actions: Se reorganizan las acciones del personalizador en una única columna. Esta modificación evita que los botones aparezcan demasiado juntos o tengan una anchura insuficiente.

.customizer__actions .common__btn-black: Se establece una anchura del 100 % y se centra el contenido mediante justify-content: center. También se ajusta el relleno interno para mejorar la interacción táctil.

.customizer__sizes: Se utiliza repeat(auto-fill, minmax(52px, 1fr)). Esta configuración permite generar automáticamente tantas columnas como quepan en el espacio disponible, garantizando que cada opción de talla tenga una anchura mínima de 52 píxeles.

.customizer__price-row: Se establece un relleno vertical de 16 píxeles para adaptar la fila del precio.

.marquee__item: Se reduce el tamaño de fuente a 12 píxeles y el espaciado entre letras a 1,5 píxeles. Esto permite que el texto animado de la marca se visualice correctamente en pantallas estrechas.

.auth__page: Se ajusta el relleno general a 20 píxeles en vertical y 12 en horizontal. También se establece align-items: flex-start para colocar el contenido al principio del eje transversal.

.auth__card: Se reduce el relleno interno de la tarjeta de autenticación a 28 píxeles verticales y 20 horizontales. Esto permite que los formularios de acceso y registro aprovechen mejor el espacio disponible.

.auth__card-title: Se establece un tamaño de fuente de 26 píxeles para adaptar el título.

.auth__form-row: Se modifica la distribución del formulario para que los campos pasen a una única columna. Se utiliza grid-template-columns: minmax(0, 1fr) y se establece una separación vertical de 20 píxeles. De esta manera, los campos del formulario pueden ocupar toda la anchura disponible.

.cart__access: Se ajusta el relleno vertical a 40 píxeles y el horizontal a 8 píxeles.

.cart__access-icon: Se reducen las dimensiones del icono de acceso al carrito a 80 × 80 píxeles.

.cart__access-actions .common__btn-black y .cart__access-actions .common__btn-white: Se establece una anchura del 100 % y se centra el contenido de los botones. Esto permite que las acciones principales puedan utilizarse cómodamente en teléfonos móviles.

.cart__item: Se modifica la distribución de los productos del carrito mediante grid-template-columns: 90px minmax(0, 1fr). Se reserva una primera columna de 90 píxeles para la imagen y se permite que la información del producto ocupe el espacio restante. También se ajusta la separación entre columnas y el relleno interno.

.cart__item-image: Se establecen unas dimensiones de 90 × 90 píxeles para la imagen de cada producto.

.cart__form-grid y .cart__card-fields: Los campos de los formularios y los datos de pago pasan a organizarse en una única columna. Esto facilita la introducción de información en pantallas estrechas.

.cart__form-section

Se reduce el relleno interno a 18 píxeles en vertical y 16 en horizontal.

.cart__payment-option: Se activa flex-wrap: wrap para permitir que los elementos de las opciones de pago pasen a una nueva línea cuando sea necesario.

.cart__summary: Se establece un relleno de 20 píxeles en vertical y 16 en horizontal para adaptar el resumen del pedido.

.newsletter: Se establece un relleno vertical de 44 píxeles para reducir el espacio ocupado por esta sección.

.newsletter__form: Se cambia la dirección del formulario a columna mediante flex-direction: column. Esto permite colocar el campo de correo electrónico y el botón de suscripción uno debajo del otro.

.newsletter__submit-btn: Se ajusta el relleno interno del botón a 16 píxeles en vertical y 20 en horizontal.

.footer__content: Se cambia la dirección de los elementos flexibles a columna y se utiliza align-items: flex-start. También se establece una separación de 20 píxeles entre los elementos. Esto permite reorganizar el contenido del pie de página verticalmente.

.footer__links: Se ajusta la separación entre los enlaces a 12 píxeles en vertical y 20 en horizontal.

.users: Se reduce el relleno general de la sección a 32 píxeles en vertical y 12 en horizontal.

.users__table th y .users__table td: Se modifica el relleno interno de las celdas de encabezado y contenido de la tabla a 14 píxeles verticales y 12 horizontales.Esto permite ajustar la presentación de la tabla a pantallas de menor tamaño.

Adaptación a móviles pequeños (360 px)

.catalog__grid: Se modifica la cuadrícula para mostrar una única columna mediante grid-template-columns: minmax(0, 1fr). En dispositivos con una anchura muy reducida, mantener dos columnas podría provocar que las tarjetas de producto resultasen demasiado estrechas. Por ello, se prioriza mostrar los productos verticalmente, permitiendo visualizar mejor sus imágenes, nombres y precios.

.header__menu: Se reduce la separación entre los enlaces del menú a 8 píxeles. Esto permite aprovechar mejor el espacio horizontal de la cabecera.

.header__link y .header__link--active: Se establece un tamaño de fuente de 13 píxeles para los enlaces normales y el enlace activo. Con ello se consigue que todos los elementos de navegación puedan visualizarse correctamente en pantallas pequeñas.

.customizer__designs: Se reduce la separación entre las opciones de diseño a 6 píxeles. Esto permite mantener una distribución compacta de los elementos del personalizador.

Capítulo 3. Sistema de Gestión Empresarial.

La siguiente parte del proyecto se encarga de la implementación de un sistema de gestión empresarial de Mercury mediante Odoo, que actúa como principal gestor de toda la actividad de la empresa. En esta parte se documenta paso a paso todo lo que se ha tenido que hacer para lograr que esté todo hecho y funcionando, desde la instalación del entorno hasta el seguimiento de un pedido personalizado de principio a fin.

3.1 Introducción.

Para el módulo de Sistemas de Gestión Empresarial se ha instalado, configurado y adaptado un entorno Odoo 19.0.Odoo representa la parte de la gestión de la empresa y todo lo que esta conlleva.

Este sistema no sustituye a la base de datos desarrollada en Acceso a Datos, sino que actúa como complemento de la misma. Los datos usados se han cargado en Odoo a partir de ficheros CSV con los mismos datos usados en el resto de proyectos pero adaptados a la base de datos de Odoo, cambiando los atributos y en algunos casos añadiendo otro tipo de identificadores. Además, los pedidos personalizados gestionados en Odoo son los que posteriormente procesa la aplicación Java del módulo de Programación de Servicios y Procesos.

3.2 Entorno.

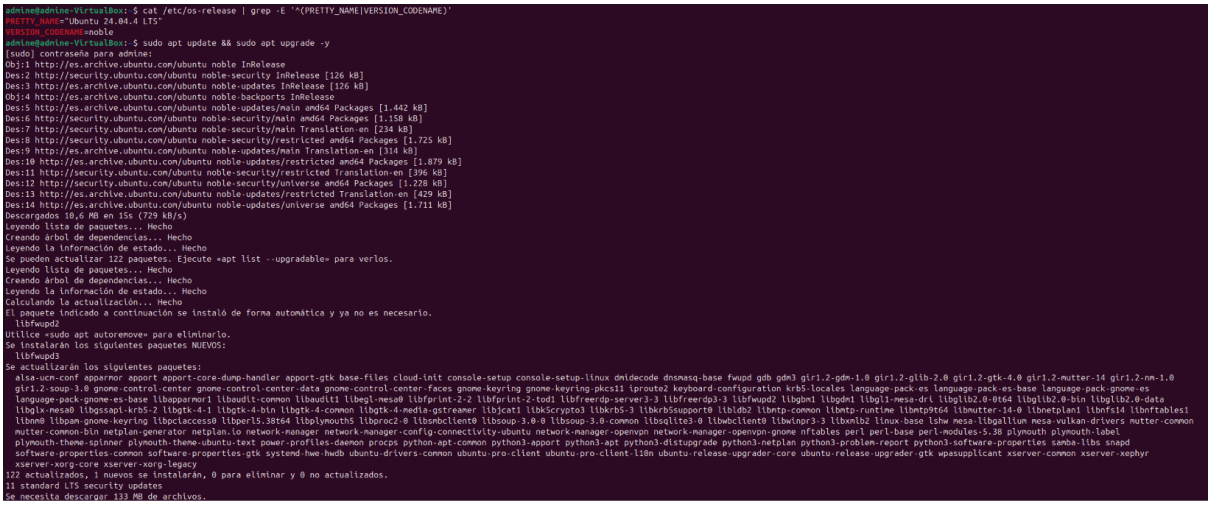
El entorno de trabajo consiste en una máquina virtual de Ubuntu 24 en VirtualBox, con las especificaciones necesarias para poder soportar la base de datos de PostgreSQL y todo lo relacionado con Odoo, siendo estas 6 GB de memoria RAM, 4 núcleos de procesador y un disco duro de 35 GB para que no hubiera ningún problema con la instalación de Ubuntu, PostgreSQL y Odoo.

3.2.1 Preparación del sistema.

El primer paso consistió en instalar las VirtualBox Guest Additions en la máquina virtual para poder copiar y pegar cosas de la máquina virtual a la máquina física y viceversa, aparte de otras herramientas útiles para hacer más sencillo el trabajo.

```
admine@admine-VirtualBox:~$ sudo /media/$USER/VBox_GAs_*/VBoxLinuxAdditions.run
Verifying archive integrity...  100%   MD5 checksums are OK. All good.
Uncompressing VirtualBox 7.2.8 Guest Additions for Linux  100%
VirtualBox Guest Additions installer
VirtualBox Guest Additions: Setting up modules
VirtualBox Guest Additions: Building the VirtualBox Guest Additions kernel
modules.  This may take a while.
VirtualBox Guest Additions: To build modules for other installed kernels, run
VirtualBox Guest Additions:   /sbin/rcvboxadd quicksetup <version>
VirtualBox Guest Additions: or
VirtualBox Guest Additions:   /sbin/rcvboxadd quicksetup all
VirtualBox Guest Additions: Building the Guest Additions 7.2.8 modules for
kernel 7.0.0-42-generic.
VirtualBox Guest Additions: Building the main module.
VirtualBox Guest Additions: Building the shared folder support module.
VirtualBox Guest Additions: Updating initramfs
update-initramfs: Generating /boot/initrd.img-7.0.0-42-generic
VirtualBox Guest Additions: Running kernel modules will not be replaced until
the system is restarted or 'rcvboxadd reload' triggered
VirtualBox Guest Additions: Starting.
VirtualBox Guest Additions: reloading kernel modules and services
VirtualBox Guest Additions: kernel modules and services 7.2.8 r173730 reloaded
VirtualBox Guest Additions: NOTE: you may still consider to re-login if some
user session specific services (Shared Clipboard, Drag and Drop, Seamless or
Guest Screen Resize) were not restarted automatically
```

A continuación se comprobó la versión del sistema operativo, que confirmó que se trabaja con Ubuntu 24.04.4 LTS. Tras esto se actualizaron la lista de paquetes y los paquetes instalados con apt update y apt upgrade.



Por último, se habilitó el repositorio universe y se volvió a actualizar la lista de paquetes. Con la instrucción apt policy se verificó que python3-pypdf2, una de las dependencias de Odoo, estuviera disponible para su instalación.

```
admine@admine-VirtualBox:~$ sudo add-apt-repository -y universe
Añadiendo componente(s) «universe» a todos los repositorios.
Obj:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Obj:4 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Leyendo lista de paquetes... Hecho
admine@admine-VirtualBox:~$ sudo apt update
Obj:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Obj:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Todos los paquetes están actualizados.
admine@admine-VirtualBox:~$ apt policy python3-pypdf2
python3-pypdf2:
  Instalados: (ninguno)
  Candidato:  2.12.1-4
  Tabla de versión:
     2.12.1-4 500
        500 http://es.archive.ubuntu.com/ubuntu noble/universe amd64 Packages
```

3.2.2 Instalación de PostgreSQL.

Odoo almacena toda su información en una base de datos PostgreSQL, por lo que este fue el primer componente en instalarse. Se utilizó apt install postgresql.

```
admine@admine-VirtualBox:~$ sudo apt install -y postgresql
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
El paquete indicado a continuación se instaló de forma automática y ya no es necesario.
  libfwupd2
Utilice «sudo apt autoremove» para eliminarlo.
Se instalarán los siguientes paquetes adicionales:
  libcommon-sense-perl libjson-perl libjson-xs-perl libllvm17t64 libpq5 libtypes-serialiser-perl postgresql-16 postgresql-client-16 postgresql-client-common postgresql-common
Paquetes sugeridos:
  postgresql-doc postgresql-doc-16
Se instalarán los siguientes paquetes NUEVOS:
  libcommon-sense-perl libjson-perl libjson-xs-perl libllvm17t64 libpq5 libtypes-serialiser-perl postgresql postgresql-16 postgresql-client-16 postgresql-client-common postgresql-common
0 actualizados, 11 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 43,6 MB de archivos.
Se utilizarán 175 MB de espacio de disco adicional después de esta operación.
```

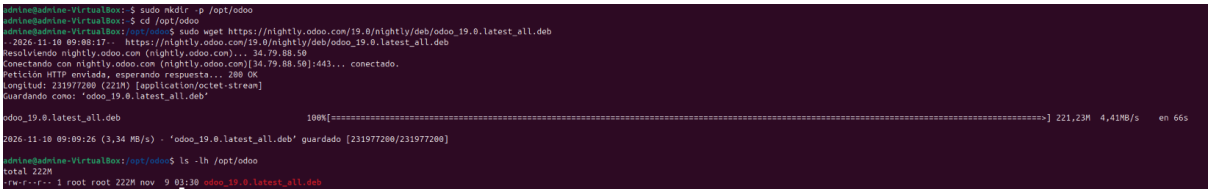
Tras la instalación se comprobó con systemctl status que el servicio postgresql estaba activo y habilitado para iniciarse automáticamente con el sistema.

```
admine@admine-VirtualBox:~$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
     Active: active (exited) since Tue 2026-11-10 09:06:30 CET; 47s ago
   Main PID: 22867 (code=exited, status=0/SUCCESS)
        CPU: 2ms

nov 10 09:06:30 admine-VirtualBox systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
nov 10 09:06:30 admine-VirtualBox systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
```

3.2.3 Instalación de Odoo.

Se creó el directorio /opt/odoo y, desde él, se descargó con wget el paquete .deb de Odoo 19.0.



El paquete se instaló con apt install indicando la ruta del fichero descargado, de forma que apt resolviera automáticamente el resto de dependencias.

```
admine@admine-VirtualBox:/opt/odoo$ sudo apt install -y ./odoo_19.0.latest_all.deb
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Nota, seleccionando «odoo» en lugar de «./odoo_19.0.latest_all.deb»
El paquete indicado a continuación se instaló de forma automática y ya no es necesario.
  libfwupd2
Utilice «sudo apt autoremove» para eliminarlo.
```

Una vez instalado, se comprobó que el servicio odoo se encontraba activo y en ejecución, y mediante el comando ss se verificó que el servidor escuchaba en el puerto 8069, que es el utilizado por defecto para acceder a la interfaz web.

```
admine@admine-VirtualBox:/opt/odoo$ sudo systemctl status odoo
● odoo.service - Odoo Open Source ERP and CRM
     Loaded: loaded (/usr/lib/systemd/system/odoo.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-11-10 09:13:18 CET; 14s ago
   Main PID: 25565 (odoo)
      Tasks: 4 (limit: 5691)
     Memory: 79.9M (peak: 80.5M)
        CPU: 627ms
     CGroup: /system.slice/odoo.service
             └─25565 /usr/bin/python3 /usr/bin/odoo --config /etc/odoo/odoo.conf --logfile /var/log/odoo/odoo-server.log

nov 10 09:13:18 admine-VirtualBox systemd[1]: Started odoo.service - Odoo Open Source ERP and CRM.
admine@admine-VirtualBox:/opt/odoo$ sudo ss -ltnp | grep 8069
LISTEN 0      128          0.0.0.0:8069       0.0.0.0:*    users:(("odoo",pid=25565,fd=4))
```

3.2.4 Instalación de Odoo.

Antes de crear la base de datos se editó el fichero de configuración /etc/odoo/odoo.conf para definir la contraseña maestra, que es la que protege las operaciones sobre las bases de datos desde la interfaz web.

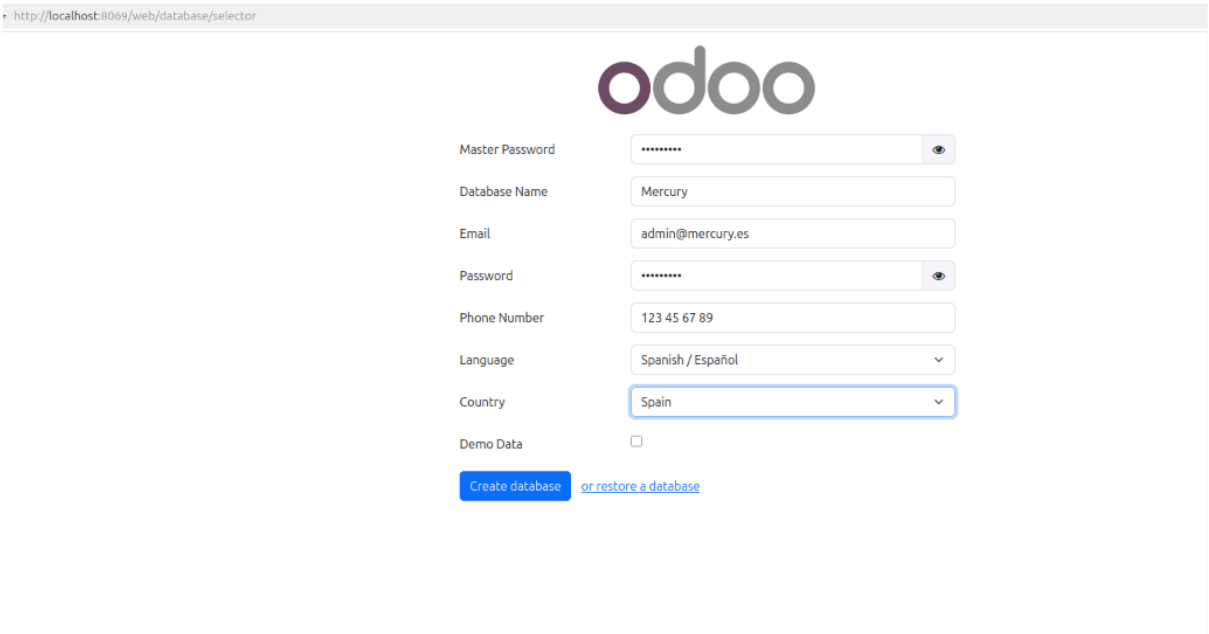
```
[options]
; This is the password that allows database operations:
; admin_passwd = admin
db_host = False
db_port = False
db_user = odoo
db_password = False
;addons_path = /usr/lib/python3/dist-packages/odoo/addons
default_productivity_apps = True
admin_passwd = Admin1234
```

Para que los cambios tuvieran efecto se reinició el servicio con systemctl restart odoo.

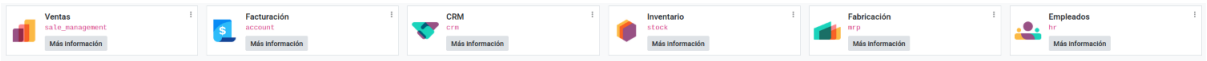
```
admine@admine-VirtualBox:/opt/odoo$ sudo systemctl restart odoo
```

3.3 Creación de la base de datos y módulos instalados.

Con el servicio en marcha, se accedió desde el navegador a localhost:8069 y se utilizó el gestor de bases de datos de Odoo para crear la base de datos de la empresa. Se le dio el nombre Mercury y luego toda la información se carga posteriormente desde los ficheros CSV del proyecto.



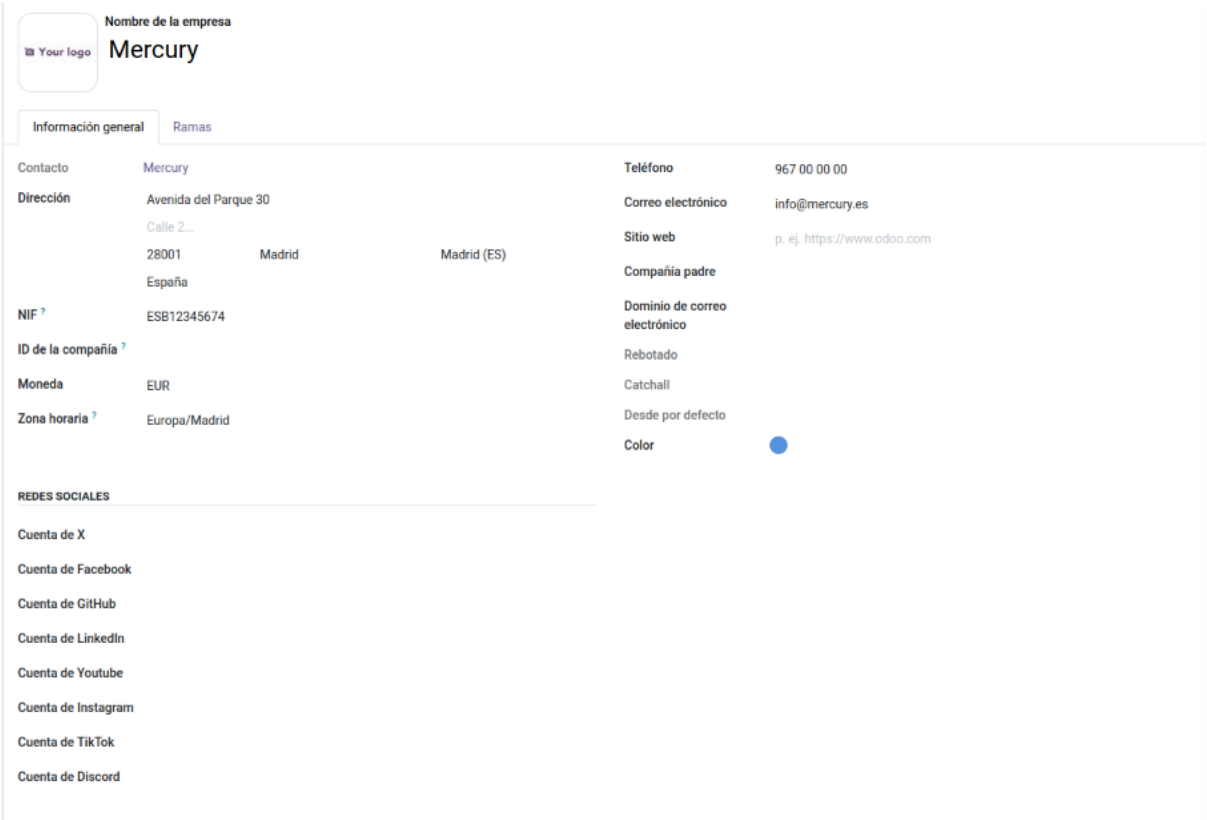
Una vez creada la base de datos, se instalaron los módulos necesarios para cubrir el flujo de la empresa: Ventas, Facturación, CRM, Inventario, Fabricación y Empleados.



3.4 Empresa, usuarios y roles.

3.4.1 Configuración de la empresa.

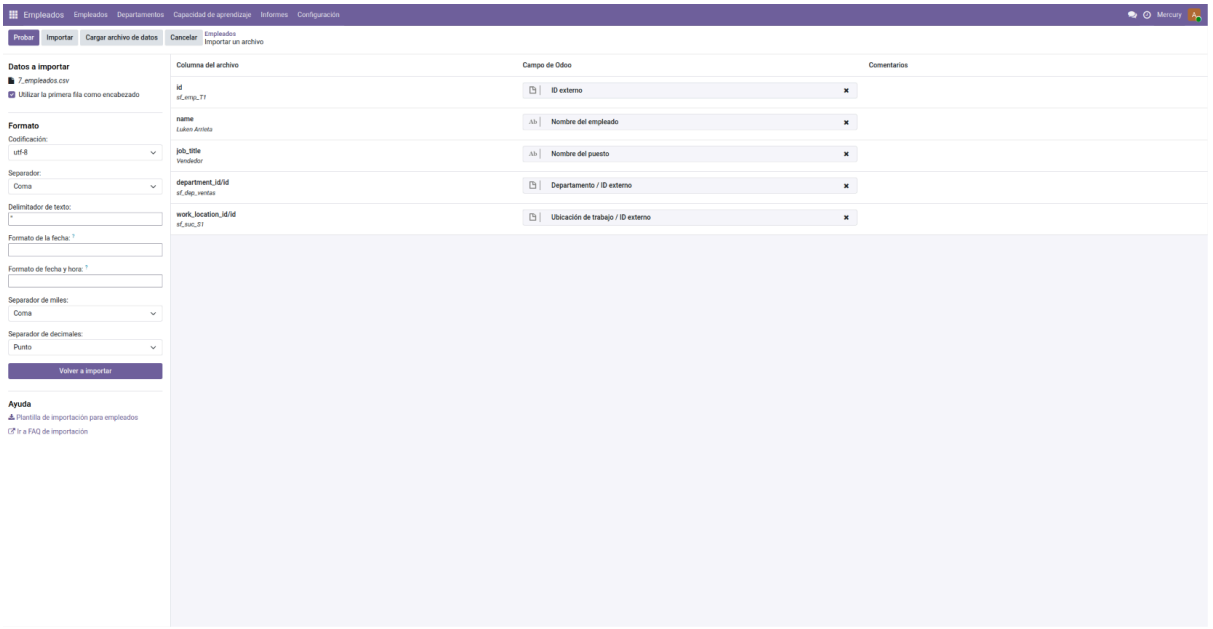
Se configuró la compañía con el nombre Mercury y sus datos, teléfono, correo de contacto info@mercury.es y NIF. Además, se estableció el euro como moneda y Europa/Madrid como zona horaria, de forma que los documentos se generen con el formato y los impuestos correspondientes a España.



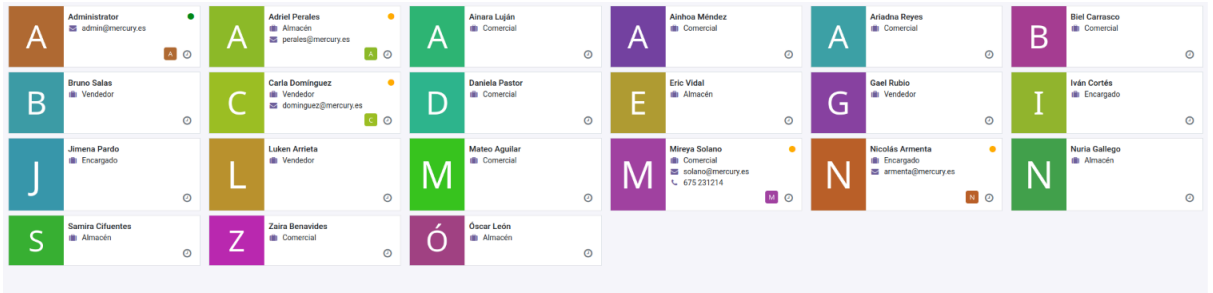
3.4.2 Importación de datos.

Los datos de la empresa se han cargado mediante la herramienta de importación de Odoo a partir de CSV. En la importación se asocian las columnas del fichero con los campos de Odoo. Se utilizó la coma como separador, y se usó la opción de probar antes de importar para detectar posibles errores.

Como ejemplo se puede ver como se han cargado los empleados de la empresa con sus respectivos atributos y sus respectivos datos.



Como resultado, el sistema muestra los 20 empleados importados junto al administrador, cada uno con su puesto, comercial, vendedor, encargado o almacén.

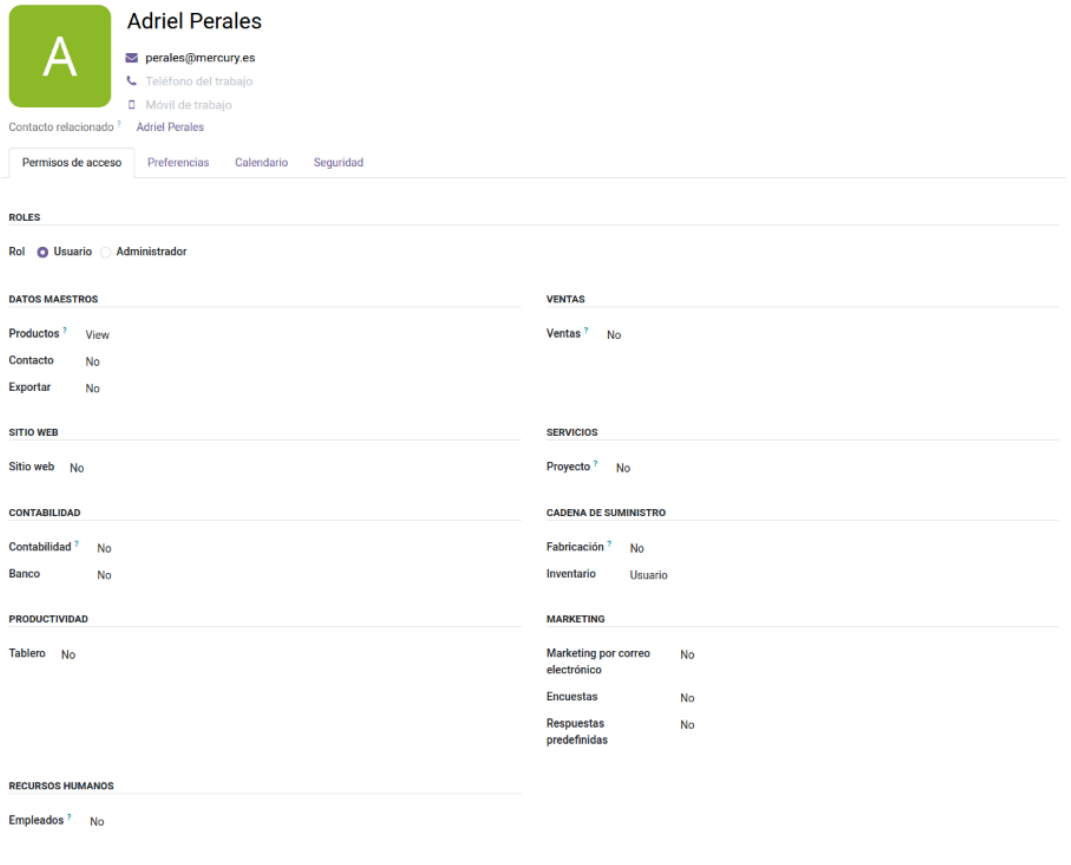


3.4.3 Usuarios y permisos.

No todos los empleados necesitan acceso al sistema, por lo que se han creado usuarios únicamente para algunos de ellos, uno por cada perfil, Adriel Perales (almacén), Carla Domínguez (vendedora), Mireya Solano (comercial) y Nicolás Armenta (encargado). Todos ellos tienen un correo de la empresa y el rol de usuario, sin privilegios de administración.

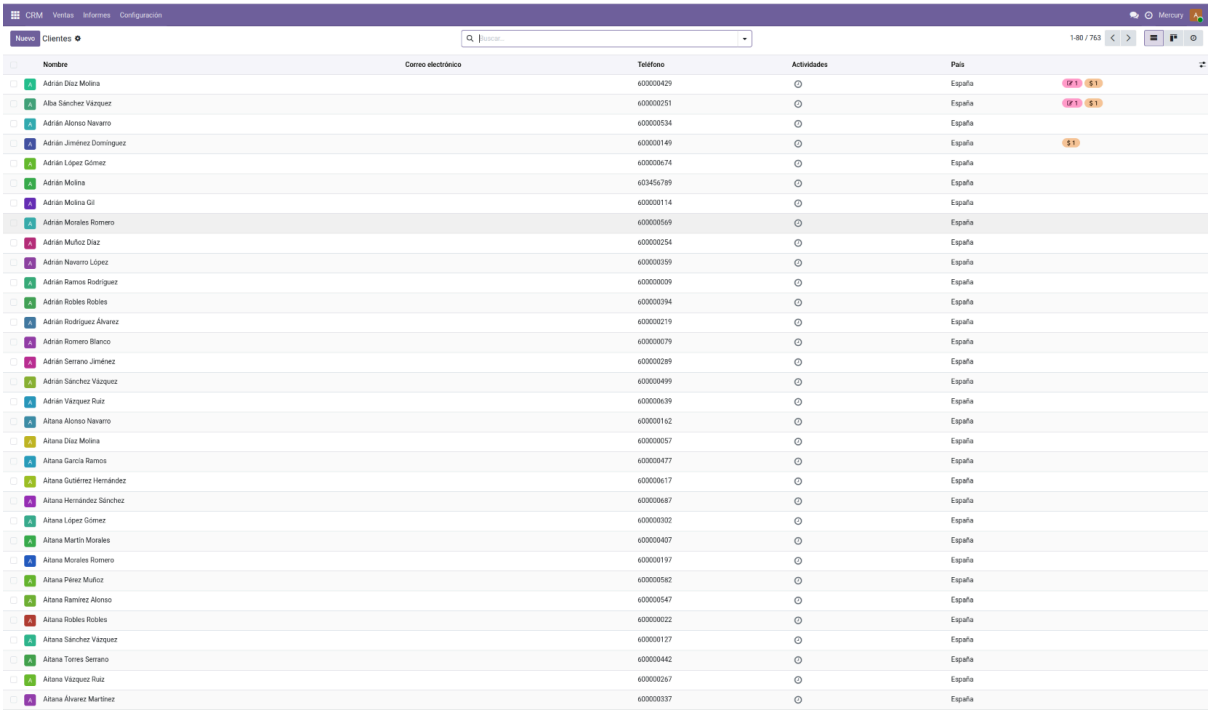


Los permisos se ajustan a la función de cada trabajador. Como ejemplo, el usuario de almacén solo puede consultar los productos, mientras que el resto de aplicaciones no tiene permisos para poder usarlas.

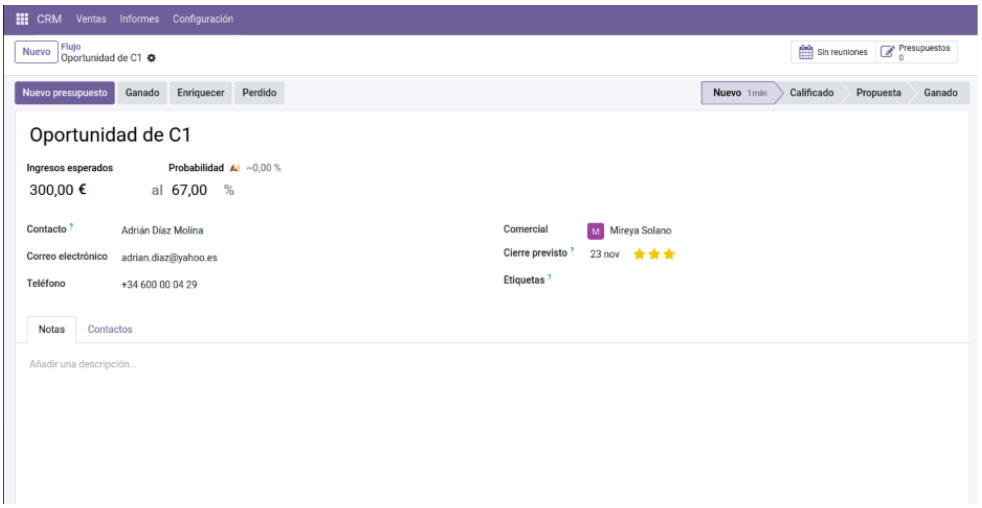


3.5 CRM y clientes.

La aplicación CRM ha quedado instalada para que los comerciales puedan registrar y seguir oportunidades antes de convertirlas en ventas. Los clientes se gestionan como contactos de Odoo, y es sobre ellos sobre los que se crean tanto las oportunidades comerciales como los pedidos de venta. En la base de datos de Odoo han sido registrados más de 700 clientes con sus respectivos datos para poder gestionar cualquier operación comercial en relación a ellos.

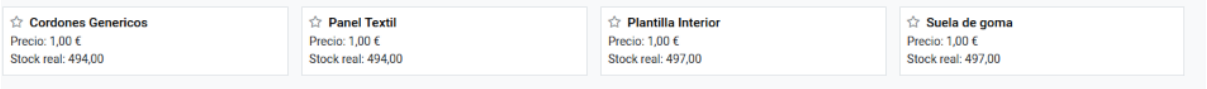


Como ejemplo del flujo previo a la venta, se ha registrado en el CRM la oportunidad *"Oportunidad de C1"* asociada al cliente Adrián Díaz Molina y asignada a la comercial Mireya Solano, con un ingreso esperado de 300,00 € y fecha de cierre prevista para el 23 de noviembre. Desde esta misma ficha es posible hacer seguimiento de la negociación y generar directamente el presupuesto una vez aceptada la propuesta.

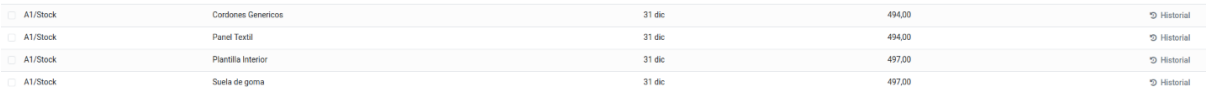


3.6 Productos, componentes e inventario.

El producto principal utilizado como ejemplo es el modelo Zapatillas Breeze Urban, aunque hay otros modelos de zapatillas y otros productos, todos ellos con sus respectivas variantes de colores y tallas. En este caso para la fabricación de las Zapatillas Breeze Urban se han dado de alta cuatro componentes, Suela de goma, Panel Textil, Plantilla Interior y Cordones Genéricos, cada uno con un coste de 1,00 € y existencias iniciales de 500 unidades.

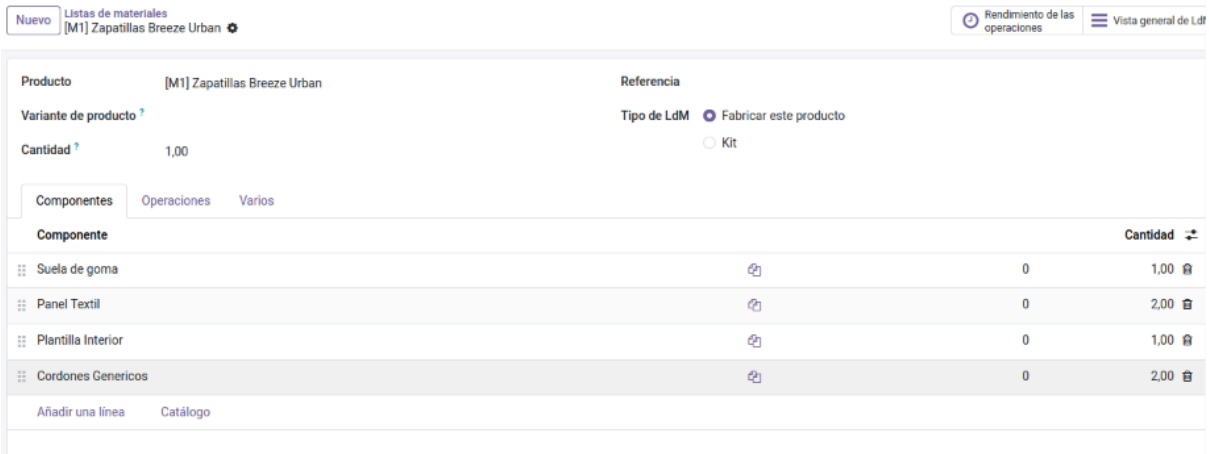


El stock de estos componentes se localiza en la ubicación A1/Stock, correspondiente al almacén A1 de Madrid, y puede consultarse desde el apartado de inventario con su historial de movimientos.



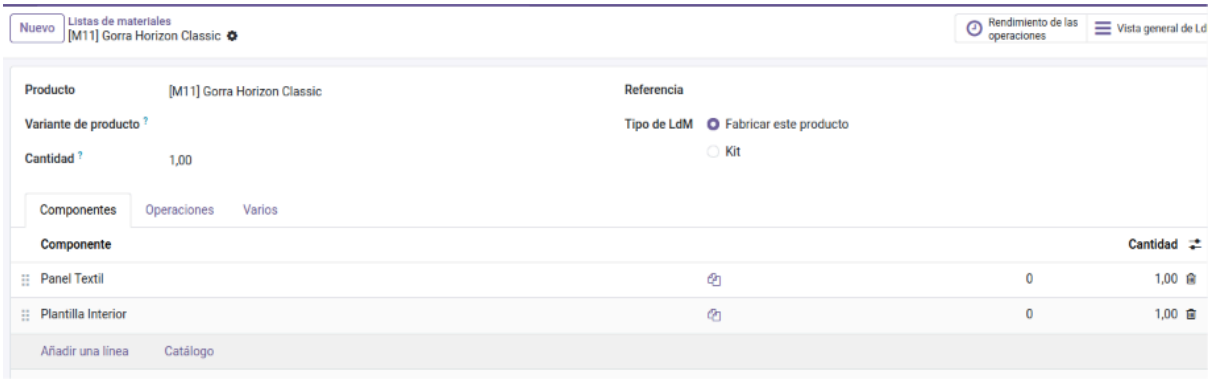
3.7 Fabricación.

Para representar los productos fabricados se ha utilizado la funcionalidad de Fabricación, definiendo una lista de materiales para algunos de los productos, sin desarrollar ningún módulo propio. En la lista de materiales del modelo Zapatillas Breeze Urban se indica que, para fabricar un par, se necesitan una suela de goma, dos paneles textiles, una plantilla interior y dos cordones.



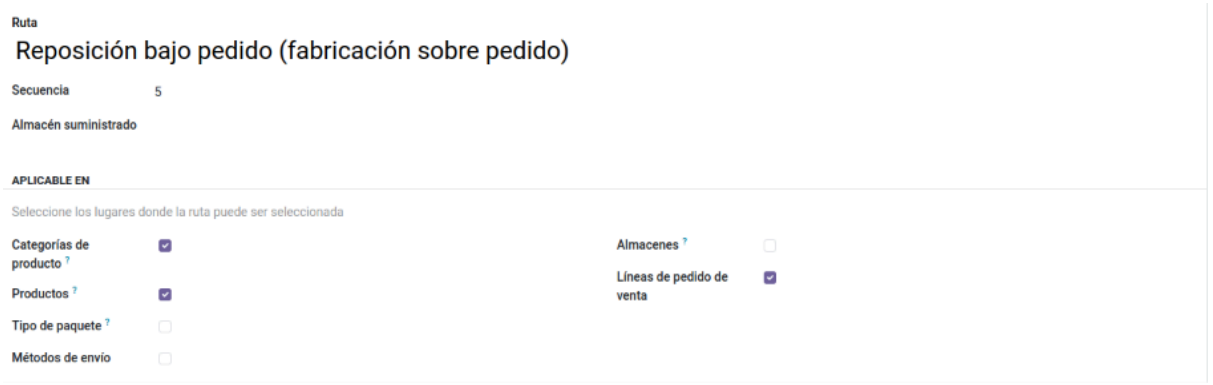
Del mismo modo, se ha definido la lista de materiales del modelo Gorra Horizon Classic, que requiere un panel textil y una plantilla interior. En ambos casos el tipo

de lista es “Fabricar este producto”, lo que permite que Odoo genere órdenes de fabricación para ellos.



3.8 Fabricación bajo pedido (MTO).

Para los productos personalizados se ha utilizado la ruta de reposición bajo pedido de Odoo. Esta ruta enlaza un pedido de venta con su fabricación, de manera que al confirmar la venta se genera de manera automática la orden de fabricación correspondiente. La ruta se ha configurado para que pueda seleccionarse en productos, categorías de producto y líneas de pedido de venta.

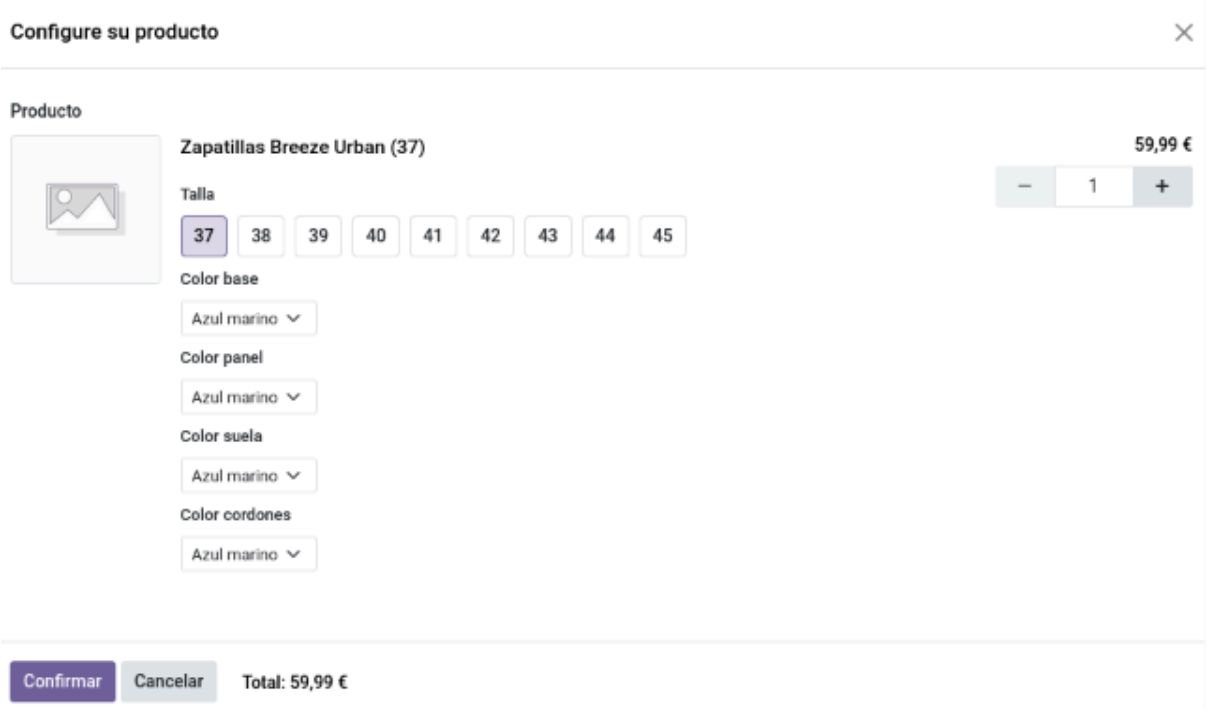


Luego se ha asignado esta ruta al producto Zapatillas Breeze Urban desde la pestaña de inventario de su ficha.



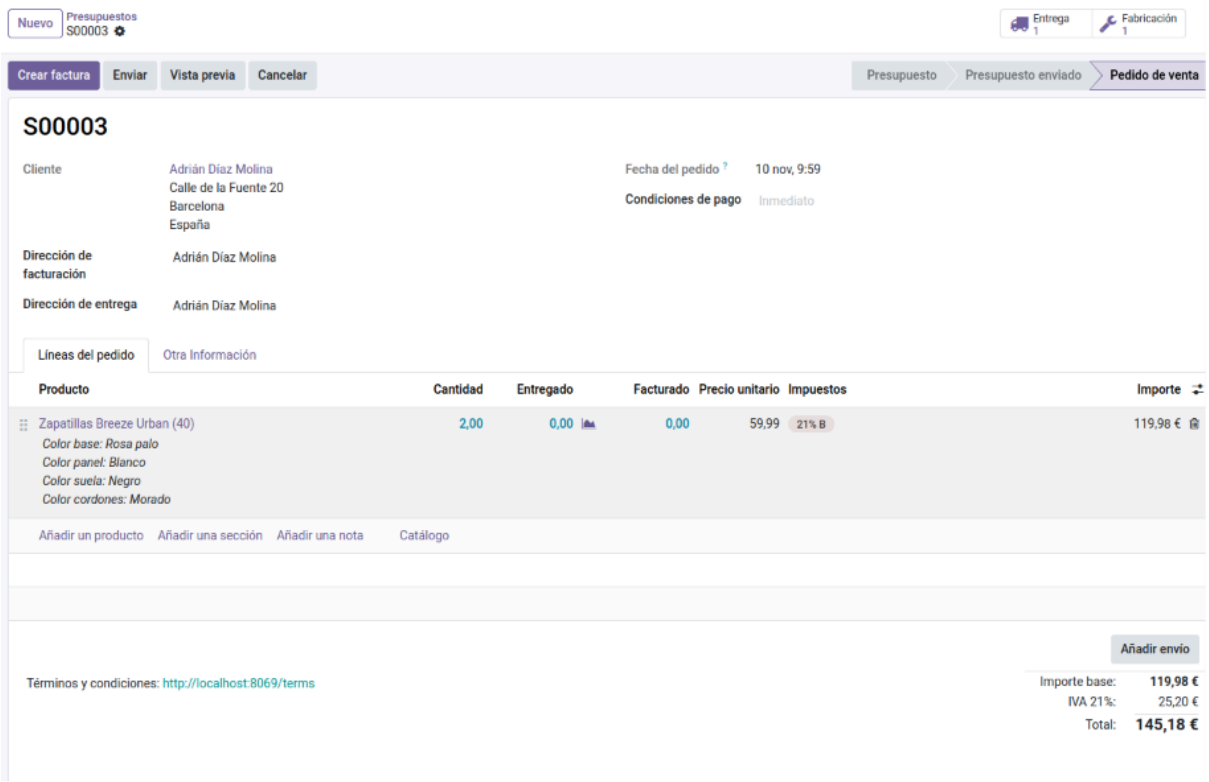
3.9 Ventas.

Al añadir el producto al presupuesto, Odoo abre el configurador de producto, en el que se selecciona la talla y el color de cada zona de la zapatilla: base, panel, suela y cordones. El configurador muestra el precio de cada unidad, 59,99 €, y el total.



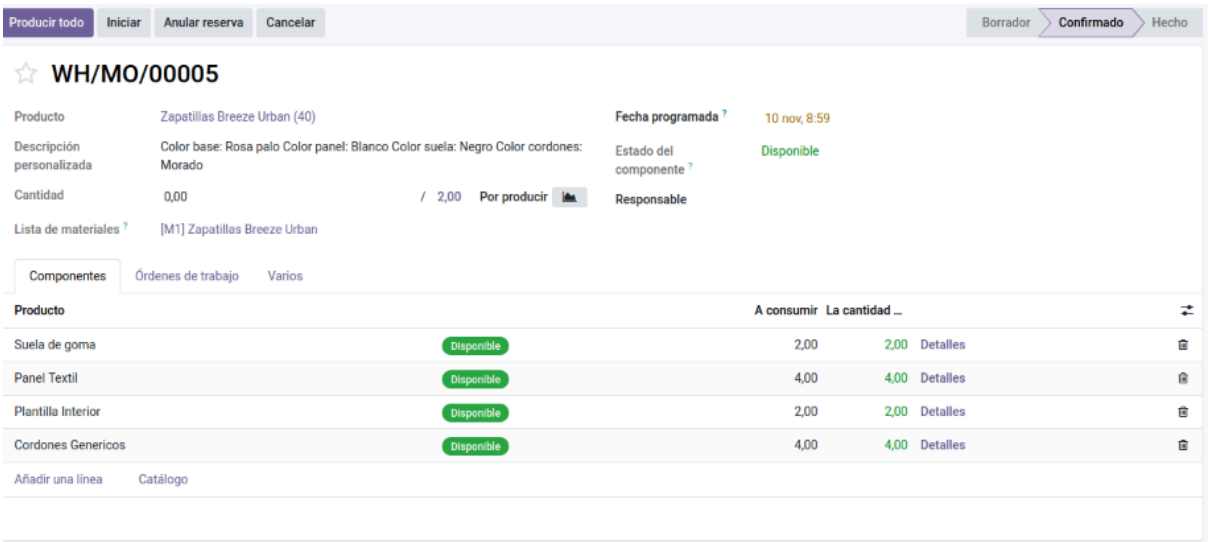
El pedido de ejemplo es el S00003, realizado por el cliente Adrián Díaz Molina, con dirección en Barcelona. Consta de dos pares de Zapatillas Breeze Urban de la talla 40, con color base rosa palo, panel blanco, suela negra y cordones morados. Estas características quedan registradas en la descripción de la línea. El precio sin IVA es de 119,98 €, al que se suma un 21 % de IVA, lo que da un total de 145,18 €. Al confirmar el presupuesto, el pedido pasa a estado de pedido de venta y aparecen

automáticamente los botones de entrega y de fabricación, gracias a la ruta MTO configurada anteriormente.

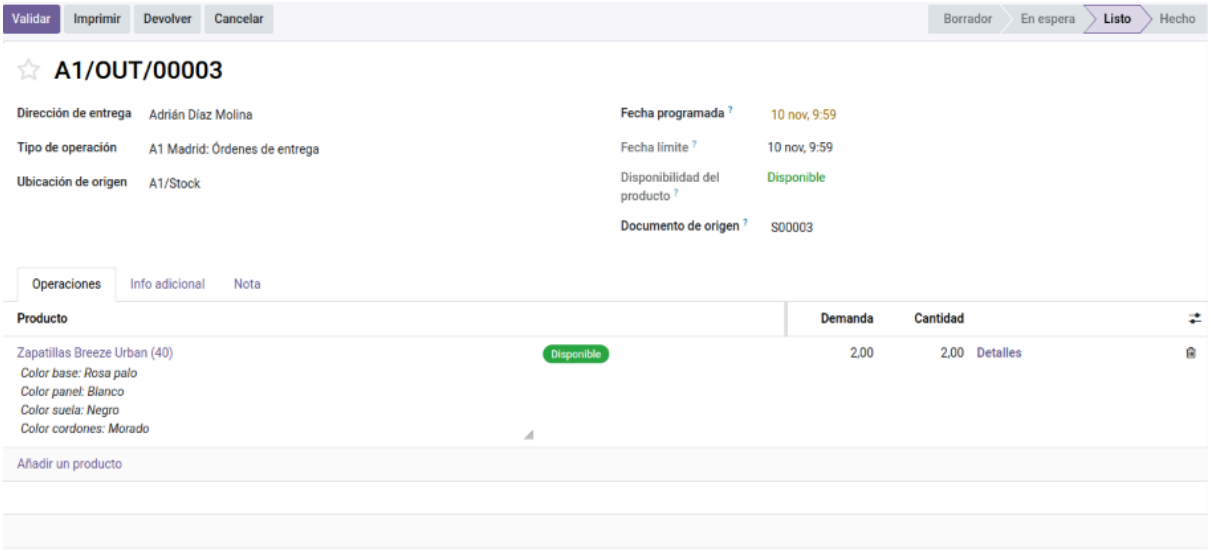


3.10 Fabricación, inventario y entrega.

Al confirmar el pedido se ha generado la orden de fabricación WH/MO/00005 para dos unidades del producto. La orden conserva la descripción personalizada con los colores elegidos y calcula los componentes necesarios, dos suelas, cuatro paneles, dos plantillas y cuatro pares de cordones, todos ellos disponibles en stock.



A la vez, el sistema ha creado la orden de entrega A1/OUT/00003. La entrega sale del almacén A1 de Madrid, hacia la dirección del cliente, e incluye la misma descripción personalizada del producto y dos unidades.

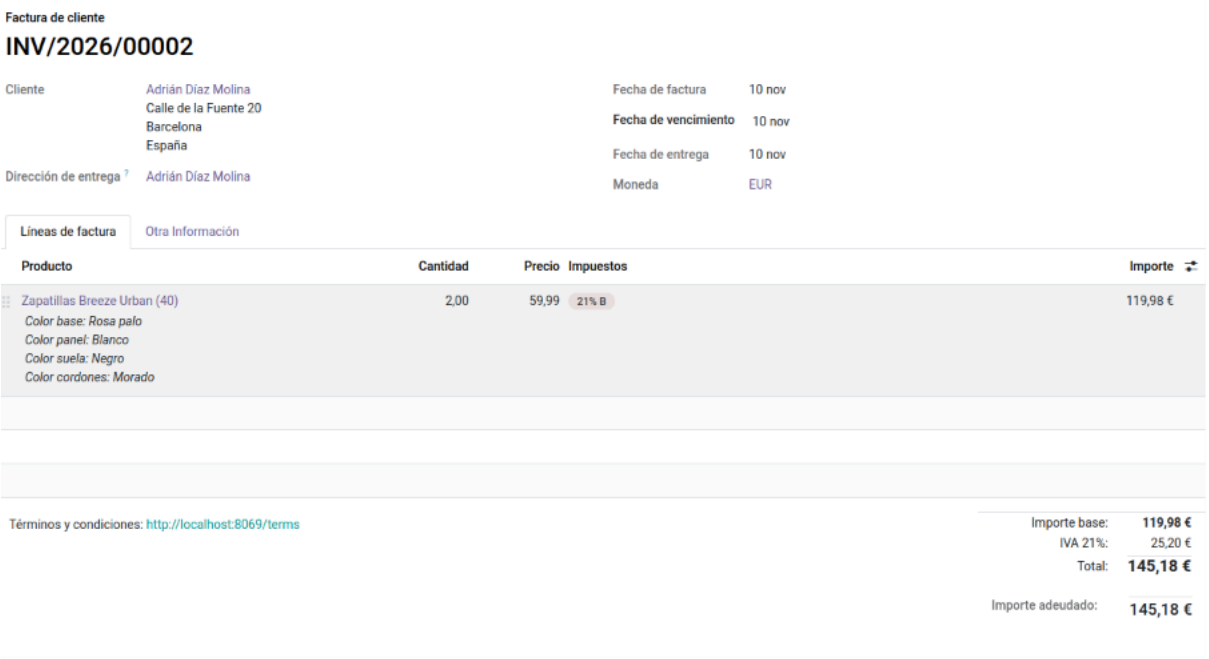


3.11 Facturación.

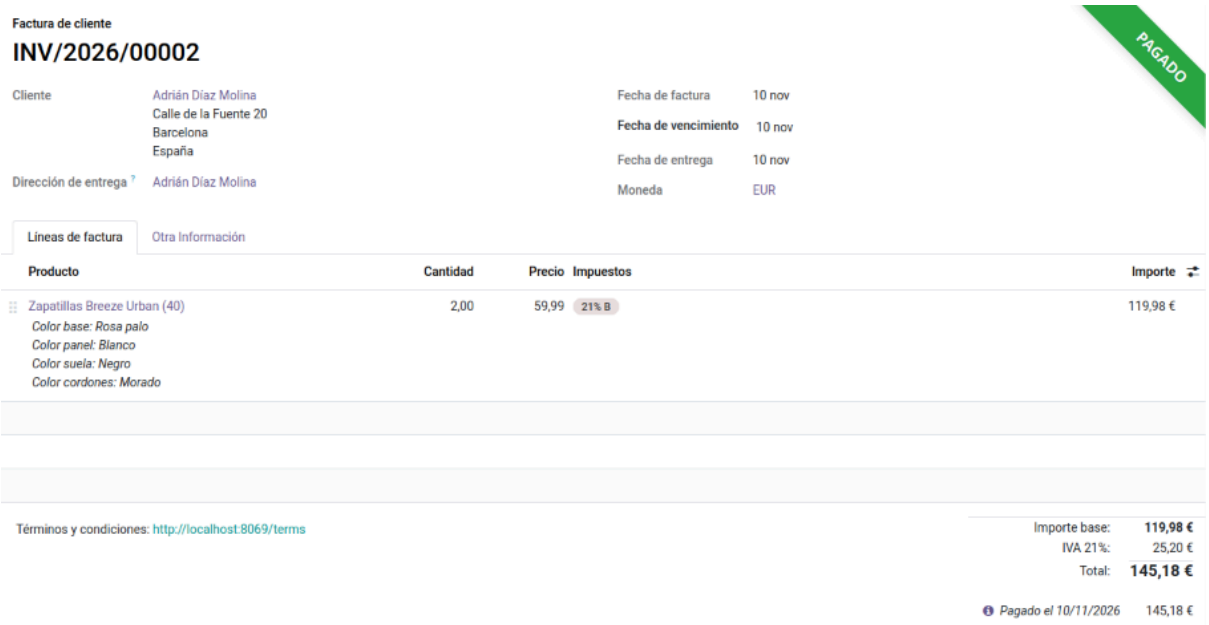
Una vez confirmado el pedido, se genera la factura desde el propio pedido de venta mediante el asistente de creación de facturas, seleccionando la opción de factura normal, ya que se factura el importe completo.



El resultado es la factura de cliente INV/2026/00002, en la que pone el producto, la cantidad, el precio, el IVA y el importe total de 145,18 €, que queda como no pagado mientras no se registra el pago.



Por último, tras registrar el pago, la factura pasa a estado pagado, indicando la fecha de pago, 10/11/2026, y el precio que el cliente ha pagado.



3.12 Trazabilidad.

El sistema permite seguir un mismo pedido desde su creación hasta que queda registrado como pagado, ya que cada documento conserva la referencia del anterior. Para el pedido de ejemplo, el recorrido completo es el siguiente:

Fase	Documento	Resultado
Venta	Pedido S00003	2 pares, total 145,18 €
Fabricación	Orden WH/MO/00005	Componentes reservados para 2 pares
Entrega	Orden A1/OUT/00003	Origen S00003, desde A1/Stock
Facturación	Factura INV/2026/00002	145,18 €, pagada el 10/11/2026

De esta forma, desde cualquiera de los documentos es posible acceder al resto mediante los botones de acceso rápido y el documento de origen, lo que permite comprobar cómo se registra y evoluciona una venta realista dentro del sistema.

En este caso, la trazabilidad es la que se ha ido mostrando durante el proceso, en las capturas de los apartados anteriores, iniciando con un presupuesto, luego un pedido que se manda a fabricar, posteriormente se procesa el envío, se crea una factura y se registra como pagado.

3.12 Resultados obtenidos.

Al finalizar este apartado se tiene un entorno Odoo completamente operativo, instalado sobre una máquina virtual de Ubuntu y configurado con los datos de la

empresa Mercury. El sistema tiene empleados y usuarios necesarios, con los permisos necesarios para cada perfil, y con los productos, componentes y listas de materiales que permiten fabricar los modelos de la colección.

Gracias a la ruta de fabricación bajo pedido, un pedido personalizado recorre de forma automática todas las fases, venta, fabricación, entrega y facturación. Esto permite gestionar de una manera sencilla los clientes, los pedidos y el stock.

Capítulo 4. Programación de Servicios y Procesos.

4.1. Planificación y metodología.

El desarrollo de este apartado del proyecto se ha centrado en la creación de una aplicación mediante Java destinada a actuar como subsistema auxiliar del ERP de Mercury. El objetivo principal de esta aplicación es complementar las funcionalidades de Odoo mediante el procesamiento de pedidos personalizados, permitiendo realizar operaciones adicionales sobre la información almacenada en el sistema de gestión empresarial.

La aplicación se ha planteado como una herramienta independiente que permite automatizar diferentes tareas relacionadas con la gestión de los pedidos, facilitando la organización de la información y proporcionando documentos que pueden utilizarse posteriormente en los procesos de fabricación. De esta manera, se pretende mejorar el tratamiento de los pedidos personalizados y reducir la necesidad de realizar determinadas comprobaciones o consultas de forma manual.

La primera fase del desarrollo consistió en analizar los requisitos establecidos para la aplicación y determinar qué funcionalidades debía incorporar. Se decidió desarrollar un programa independiente que recibiese una exportación de los datos de un pedido personalizado y se encargará de coordinar diferentes procesos para realizar tareas complementarias sobre dicha información.

Una vez establecidas las funcionalidades, se planteó una estructura basada en un proceso principal encargado de coordinar la ejecución de varios procesos secundarios. Para ello, se decidió utilizar la clase ProcessBuilder de Java, que permite iniciar otros programas o procesos desde la aplicación principal.

Posteriormente, se organizó el trabajo de los distintos procesos especializados, asignando a cada uno una responsabilidad concreta dentro del tratamiento del pedido. También se planificó el mecanismo de comunicación entre los procesos, así como el control de los posibles errores durante su ejecución.

4.2. Descripción de la solución.

La solución desarrollada consiste en una aplicación Java que funciona como servicio auxiliar al sistema de gestión de Mercury. Su función es recibir los datos de un pedido personalizado y coordinar diversos procesos independientes para realizar operaciones complementarias sobre dicho pedido.

El objetivo de la solución desarrollada es transformar la información original del pedido en diferentes resultados que se puedan utilizar en otras áreas de trabajo. En lugar de limitarse a leer los datos del archivo CSV, se realizan comprobaciones sobre su contenido así como organizar las cantidades que se han solicitado y generar documentos para facilitar la consulta y preparación de los pedidos.

Su funcionamiento comienza cuando el programa principal recibe los datos necesarios para procesar un pedido. Estos datos incluyen la ruta del archivo CSV, el número total de pares solicitados y el rango de tallas permitido. Dicho archivo contiene información sobre elementos como el pedido, el cliente o los detalles del diseño personalizado.

A partir de esta información, el programa coordinador inicia los distintos procesos encargados de realizar las operaciones necesarias. El primero de ellos es el proceso validador que comprueba que los datos sean correctos antes de continuar. Si la validación termina correctamente se ejecutan los procesos encargados de generar la ficha para el taller y los resúmenes de las tallas.

La aplicación también incorpora una serie de mecanismos destinados a controlar la ejecución de los programas secundarios. Entre ellos se encuentran la comprobación de los códigos de salida, la captura de los mensajes de error y el control del tiempo máximo permitido para cada ejecución.

Además, se genera un registro de actividad que permite consultar posteriormente cómo se ha desarrollado el procesamiento del pedido. Esto resulta útil para identificar posibles problemas y conocer qué operaciones se han completado correctamente.

4.2.1 Proceso central.

El elemento en el que se centra toda la aplicación, no es otro que la clase Coordinador, encargada de controlar todo el procesamiento del pedido. El programa empieza leyendo el fichero CSV proporcionado como argumento y obteniendo el identificador del pedido. Tras esto se crea una carpeta donde se almacenarán todos los archivos que se generen durante la ejecución del programa.

Para iniciar los procesos secundarios se utiliza ProcessBuilder para poder ejecutar cada uno de los procesos necesarios.

El proceso coordinador se encarga de enviar el contenido del pedido a los procesos secundarios para que estos realicen las operaciones necesarias. Y se encarga de recoger sus salidas y los posibles errores que se generen, mostrando los resultados por consola y permitiendo así registrar el estado de cada ejecución.

4.2.2 Procesos secundarios.

El programa desarrollado en Java cuenta con cuatro procesos secundarios, encargados de realizar distintas operaciones y creados cada uno en una clase de Java independiente.

Proceso validador

Dicho proceso es el encargado de comprobar que los datos recibidos del proceso central cumplen las condiciones necesarias para poder procesar el pedido personalizado. Es especialmente importante porque determina si se puede continuar hacia la siguiente fase de las operaciones.

Primero comprueba que los argumentos proporcionados sean correctos y que el número total de pares y el rango de tallas sean válidos. Posteriormente, analiza el fichero CSV para verificar que su cabecera tenga la estructura esperada y que cada registro contenga los seis campos necesarios. Además de comprobar que no existan campos vacíos, que las tallas sean valores numéricos y que las cantidades sean números positivos.

Por último, calcula el número total de pares del pedido y lo compara con el valor esperado. Si dicha comprobación es satisfactoria, devuelve un código de salida 0, pero si se encuentran errores en los datos devuelve un código 1, mientras que si los errores se relacionan con los argumentos o el procesamiento el código sería 2.

```java
if (totalPares != totalEsperado) {
    errores.add("El total de pares es " + totalPares + " y se esperaban " + totalEsperado);
}

if (errores.isEmpty()) {
    System.out.println("VALIDO: pedido correcto con " + totalPares + " pares");
    System.exit( status: 0);
} else {
    System.out.println("NO VALIDO: " + errores.size() + " errores encontrados");
    for (String error : errores) {
        System.err.println(error);
    }
    System.exit( status: 1);
}
```

```java
try {
    totalEsperado = Integer.parseInt(args[0]);
    tallaMin = Integer.parseInt(args[1]);
    tallaMax = Integer.parseInt(args[2]);
} catch (NumberFormatException e) {
    System.err.println("ERROR: los argumentos deben ser números");
    System.exit( status: 2);
}

if (totalEsperado <= 0 || tallaMin > tallaMax) {
    System.err.println("ERROR: total o rango de tallas incorrecto");
    System.exit( status: 2);
}
```

Proceso Resumen de tallas

Este proceso se encarga de analizar las cantidades solicitadas y agruparlas según las diferentes tallas de los productos. Su finalidad es transformar los datos originales del CSV en un resumen más sencillo de consultar, permitiendo conocer rápidamente cuántos pares se necesitan para cada talla.

Para realizar esta operación, se utiliza una estructura de TreeMap, que permite almacenar las tallas junto a sus respectivas cantidades y ordenarlas de forma numérica. El programa empieza a crear un TreeMap donde se almacenan los resultados y el número total de pares. A continuación se lee la información recibida utilizando un BufferedReader.

A medida que se leen los registros del fichero CSV, se acumulan las cantidades correspondientes a cada talla. Una vez finalizado el procesamiento, se recorre el contenido del TreeMap y se muestra un resumen de pares por tallas. Gracias a este proceso, se permite obtener una visión más clara de las cantidades que se deben preparar para cada una de las distintas tallas disponibles.

```java
Map<Integer, Integer> tallas = new TreeMap<>();
int total = 0;

try {
    BufferedReader entrada = new BufferedReader(new InputStreamReader(System.in, StandardCharsets.UTF_8))
    entrada.readLine();
    String linea;

    while ((linea = entrada.readLine()) != null) {
        if (linea.isBlank()) continue;
        String[] campos = linea.split( regex: ";",  limit: -1);

        int talla = Integer.parseInt(campos[3].trim());
        int cantidad = Integer.parseInt(campos[4].trim());

        tallas.merge(talla, cantidad, Integer::sum);
        total += cantidad;
    }
```

Proceso Creador de Fichas

Este proceso se corresponde con la clase FichaTaller y tiene como objetivo generar un documento con la información necesaria para preparar un pedido personalizado. Para ello, procesa los datos del CSV y extrae información sobre el diseño seleccionado, el modelo o el cliente y agrupa las cantidades solicitadas por talla, utilizando una vez más la estructura TreeMap.

```java
try {
    BufferedReader entrada = new BufferedReader(new InputStreamReader(System.in, StandardCharsets.UTF_8));
    entrada.readLine();
    String linea;

    while ((linea = entrada.readLine()) != null) {
        if (linea.isBlank()) continue;
        String[] campos = linea.split( regex: ";",  limit: -1);

        if (pedido.isEmpty()) {
            pedido = campos[0].trim();
            cliente = campos[1].trim();
            modelo = campos[2].trim();
            diseno = campos[5].trim();
        }

        int talla = Integer.parseInt(campos[3].trim());
        int cantidad = Integer.parseInt(campos[4].trim());
        tallas.merge(talla, cantidad, Integer::sum);
        total += cantidad;
    }
```

Una vez recopilada toda la información, se genera un fichero de texto que contiene una ficha organizada con los datos principales del pedido, la fecha de generación, las cantidades por talla y el número total de pares. El documento generado se almacena en la carpeta salida con el nombre correspondiente al identificador del pedido. De esta manera, se obtiene un archivo que facilita la consulta de la información necesaria para el trabajo del taller.

```
List<String> ficha = new ArrayList<>();
ficha.add("FICHA DE TALLER");
ficha.add("Fecha: " + LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm")));
ficha.add("Pedido: " + pedido);
ficha.add("Cliente: " + cliente);
ficha.add("Modelo: " + modelo);
ficha.add("Diseño: " + diseno);
ficha.add("");
ficha.add("Talla | Pares");

for (Map.Entry<Integer, Integer> t : tallas.entrySet()) {
    ficha.add(String.format("%5d | %5d", t.getKey(), t.getValue()));
}
ficha.add("TOTAL | " + String.format("%5d", total));

try {
    Files.write(rutaFicha, ficha, StandardCharsets.UTF_8);
} catch (IOException e) {
    System.err.println("ERROR: no se pudo guardar la ficha en " + rutaFicha);
    System.exit( status: 2);
}

System.out.println("Ficha generada en " + rutaFicha);
System.exit( status: 0);
```

Proceso Creador de Informe

Este proceso se consigue mediante la clase informe, que es la encargada de generar un documento que recoge los resultados obtenidos durante el procesamiento del pedido. Para ello, recibe la información proporcionada por el proceso coordinador, incluyendo los resultados de los procesos Validador, ResumenTallas y FichaTaller.

```
String datosInforme = "Pedido " + idPedido + " (" + rutaPedido + ")\n\n"
        + bloque( clase: "Validador", validacion) + "\n"
        + bloque( clase: "ResumenTallas", resumen) + "\n"
        + bloque( clase: "FichaTaller", ficha);
Resultado informe = ejecutarHijo( clase: "Informe", datosInforme, rutaInforme.toString());
```

Primero se establece la ruta donde se deberá guardar el informe y se crea una lista de cadenas de texto que almacenará las distintas partes del documento, donde se almacenará el título del informe, la fecha y hora de generación y el contenido del mismo. Tras esto, el proceso comienza a leer la información que recibe mediante la entrada estándar. Esta información es enviada por el proceso coordinador y contiene los resultados de los procesos Validador, ResumenTallar y FichaTaller.

Con estos datos se genera un fichero de texto que incluye un título, la fecha y hora de generación y los resultados de las diferentes operaciones realizadas. El contenido recibido se incorpora a la lista de cadenas de texto creada anteriormente.

```
List<String> informe = new ArrayList<>();
informe.add("INFORME DE PROCESAMIENTO");
informe.add("Generado: " + LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss")));
informe.add("");

try {
    BufferedReader entrada = new BufferedReader(new InputStreamReader(System.in, StandardCharsets.UTF_8));
    String linea;
    while ((linea = entrada.readLine()) != null) {
        informe.add(linea);
    }
    Files.write(rutaInforme, informe, StandardCharsets.UTF_8);

} catch (IOException e) {
    System.err.println("ERROR: no se pudo generar el informe en " + rutaInforme);
    System.exit( status: 2);
}

System.out.println("Informe generado en " + rutaInforme);
System.exit( status: 0);
```

Una vez obtenida toda la información se utiliza el método Files.write para guardar en documento en la ruta indicada, utilizando una codificación UTF-8. El informe se almacena en la carpeta salida, permitiendo consultar posteriormente cómo se ha desarrollado el procesamiento del pedido.

4.2.3 Comunicación entre procesos.

Uno de los aspectos más importantes de esta aplicación es la comunicación entre el proceso principal y los procesos secundarios. Para realizar esta comunicación se utilizan los flujos estándar de entrada y salida proporcionados por Java. Este mecanismo permite que los diferentes programas intercambien información sin necesidad de compartir directamente sus variables o estructuras de datos internas. Cada proceso funciona de forma independiente y utiliza los canales de comunicación establecidos para recibir los datos que necesita y devolver los resultados de su ejecución-

El proceso coordinador envía los datos del pedido a través de la entrada estándar de los procesos secundarios, utilizando un BufferedWriter conectado al flujo de entrada del proceso. De esta manera, los programas especializados pueden recibir y leer la información mediante BufferedReader.

En el programa esta comunicación se realiza dentro del método ejecutarHijo. Una vez iniciado el proceso, se obtiene su flujo de entrada a través del método getOutputStram. Sobre dicho flujo se utiliza OutputStreamWriter y un BufferedWriter para facilitar el envío del contenido del pedido. Por su parte los procesos secundarios utilizan BufferedReader para leer la información recibida. De esta forma, se permite procesar el contenido línea a línea lo que facilita la

separación de la cabecera del archivo con los registros del CSV para así evitar posibles problemas.

Por otro lado, la salida estándar y la salida de errores de cada proceso se redirigen a ficheros temporales mediante ProcessBuilder. Una vez finalizada la ejecución, el coordinador lee estos archivos y recupera la información generada.

Gracias a este mecanismo se consigue separar la ejecución de cada programa y mantener un intercambio de información organizado entre los distintos componentes de la aplicación.

4.2.4 Control de ejecución y gestión de errores.

La aplicación incorpora diferentes mecanismos para controlar la ejecución de los procesos y gestionar incidencias que puedan ocurrir a lo largo del desarrollo del programa.

En primer lugar, el coordinador comprueba el código de salida de cada proceso para determinar si ha terminado correctamente o si se ha producido algún error. Además de establecer un tiempo de espera máximo para cada proceso secundario, siendo este de diez segundos.

En caso de que algún proceso supere dicho límite, se fuerza su finalización mediante el método destroyForcibly y se registra la incidencia como un tiempo de espera agotado. Además se contemplan otro tipo de errores, como que se relacionen con la imposibilidad de iniciar un proceso, posibles interrupciones durante la espera o problemas en la lectura de los ficheros.

```
boolean terminado = proceso.waitFor(TIMEOUT_SEGUNDOS, TimeUnit.SECONDS);
if (terminado) {
    r.codigo = proceso.exitValue();
} else {
    proceso.destroyForcibly();
    proceso.waitFor();
    r.timeout = true;
    r.codigo = -1;
}
```

Por último, en la aplicación se encuentra implementado un sistema de registro de actividad mediante una lista de mensajes que almacena las operaciones realizadas junto con su hora de ejecución. Al finalizar el procesamiento, esta información se guarda en un fichero de texto dentro de la carpeta de salida, de esta forma se puede consultar el estado de cada proceso y conocer si el pedido se ha procesado correctamente o si se han producido incidencias.

```
private static void guardarLog(Path rutaLog) {  2 usages
    try {
        Files.write(rutaLog, log, StandardCharsets.UTF_8);
        System.out.println("Log guardado en " + rutaLog);
    } catch (IOException e) {
        System.err.println("ERROR: no se pudo guardar el log en " + rutaLog);
    }
}
```

4.3 Archivos generados

Durante la ejecución del programa se procesan diversos pedidos y como resultado del procesamiento de cada uno de ellos, la aplicación genera diferentes archivos que permiten consultar la información obtenida. Dichos archivos se almacenan dentro de la carpeta de salida, utilizando el identificador del pedido para distinguirlos.

El primer archivo es la ficha del pedido, que contiene la información del mismo y un resumen de las cantidades necesarias, que va destinado al taller donde se fabrican las zapatillas. El siguiente fichero es el informe, donde se recogen los resultados de cada uno de los procesos ejecutados y la fecha de generación de los mismos. Por

último, el archivo log, permite mantener un registro de las distintas operaciones realizadas así como sus códigos de salida y posibles errores.

4.4 Resultados Obtenidos

Al finalizar el desarrollo de este apartado se ha obtenido una aplicación de consola desarrollada en Java capaz de procesar pedidos personalizados mediante la coordinación de diferentes procesos independientes.

El resultado principal ha sido la implementación de un programa coordinador que recibe los datos de un pedido en formato CSV y utiliza ProcessBuilder para iniciar los programas especializados encargados de realizar las distintas operaciones.

Gracias a la implementación del proceso Validador, se ha conseguido incorporar un sistema de comprobación de los datos que permite detectar errores en la estructura del fichero, las tallas, las cantidades y el número total de pares solicitados. De esta manera, se evita continuar con el procesamiento cuando un pedido no cumple las condiciones establecidas.

Por otro lado, los procesos ResumenTallas y FichaTaller permiten transformar los datos originales del pedido en información más organizada y útil. El primero genera un resumen de las cantidades agrupadas por talla, mientras que el segundo crea una ficha de texto con los datos necesarios para preparar el pedido personalizado.

Además, mediante el proceso Informe se obtiene un documento que reúne los resultados de las diferentes operaciones realizadas. A esto se añade el sistema de registro de actividad, que permite consultar los tiempos de ejecución, los códigos de salida y las posibles incidencias.

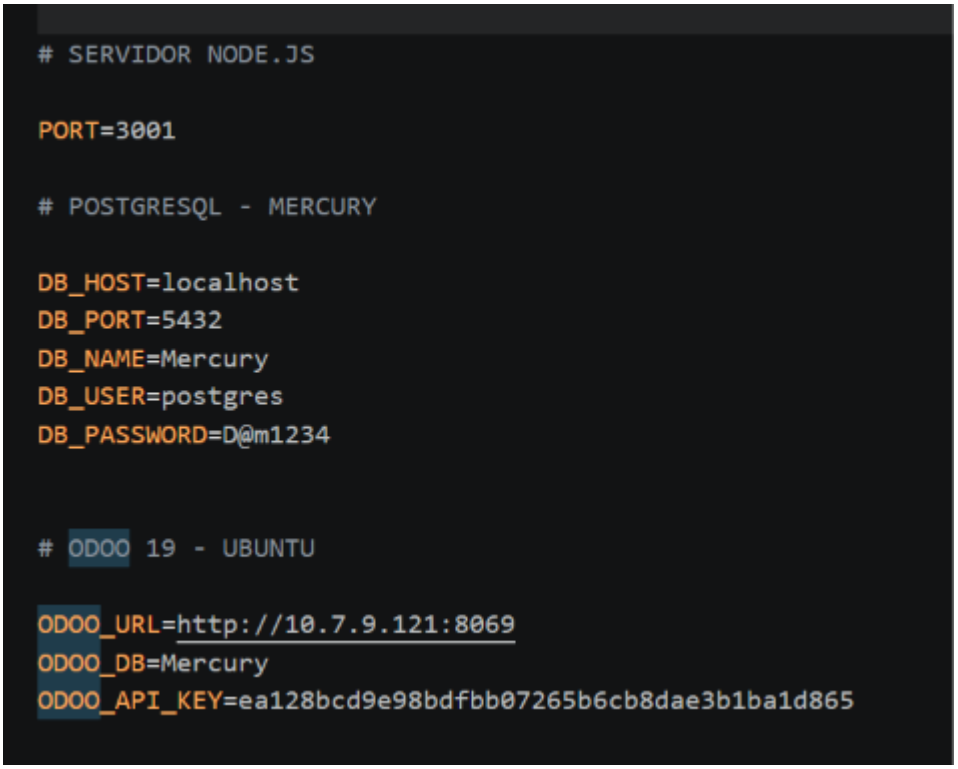
Otro de los resultados importantes ha sido la aplicación de diferentes mecanismos de comunicación y control entre procesos, utilizando los flujos de entrada y salida estándar, la gestión de códigos de finalización y el establecimiento de tiempos máximos de ejecución.

En conclusión, el desarrollo de este apartado ha permitido aplicar de forma práctica los conocimientos adquiridos en el módulo de Programación de Servicios y Procesos, especialmente los relacionados con la creación y coordinación de procesos externos mediante Java. Además, se ha conseguido desarrollar una herramienta complementaria al ERP de Mercury que facilita el tratamiento de los pedidos personalizados sin sustituir las funcionalidades principales de Odoo.

Capítulo 5. Gestión de errores.

Durante la fase final del proyecto surgieron dificultades al intentar integrar Odoo 19, utilizado como sistema ERP, con la base de datos externa desarrollada en PostgreSQL, que da soporte a la aplicación web. El objetivo inicial era establecer una comunicación automática entre ambos sistemas para sincronizar la información de los pedidos y mantener los datos actualizados en las dos plataformas.

Ante la imposibilidad de realizar una conexión directa entre las bases de datos sin comprometer la integridad de las tablas internas de Odoo, se optó por desarrollar una API intermedia. Para configurar esta conexión de forma segura, se definieron las variables de entorno en un archivo .env con los parámetros de acceso tanto a la base de datos local de PostgreSQL como a la instancia de Odoo 19 alojada en el servidor Ubuntu , incluyendo el nombre de la base de datos y la clave de acceso.



```
# SERVIDOR NODE.JS

PORT=3001

# POSTGRESQL - MERCURY

DB_HOST=localhost
DB_PORT=5432
DB_NAME=Mercury
DB_USER=postgres
DB_PASSWORD=D@m1234


# ODOO 19 - UBUNTU

ODOO_URL=http://10.7.9.121:8069
ODOO_DB=Mercury
ODOO_API_KEY=ea128bcd9e98bdfbb07265b6cb8dae3b1ba1d865
```

A nivel de desarrollo, se implementó la función asíncrona consultarOdoo encargada de realizar peticiones HTTP POST contra los endpoints de la API JSON-2 de Odoo. Esta función envía los parámetros en formato JSON dentro del cuerpo de la petición y gestiona la autenticación mediante la cabecera Authorization: Bearer junto con X-Odoo-Database para indicar la base de datos destino. A partir de esta función base, se crearon rutas REST en Express como /api/odoo/estado y /api/odoo/clientes.

```
/* ODOO 19 - API JSON-2 */

const ODOO_URL = process.env.ODOO_URL;
const ODOO_DB = process.env.ODOO_DB;
const ODOO_API_KEY = process.env.ODOO_API_KEY;

async function consultarOdoo(modelo, metodo, datos = {}) {

    if (!ODOO_URL || !ODOO_API_KEY) {
        throw new Error("Falta configurar ODOO_URL u ODOO_API_KEY");
    }

    const respuesta = await fetch(
        `${ODOO_URL.replace(/\/$/, "")}/json/2/${modelo}/${metodo}`,
        {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
                "Authorization": `bearer ${ODOO_API_KEY}`,
                ...(ODOO_DB ? { "X-Odoo-Database": ODOO_DB } : {})
            },
            body: JSON.stringify(datos)
        }
    );

    if (!respuesta.ok) {
        const error = await respuesta.text();
        throw new Error(`Error API Odoo (${respuesta.status}): ${error}`);
    }

    return await respuesta.json();
}
```

```
/* API - ODOO */

app.get("/api/odoo/estado", async (req, res) => {

    try {
        const resultado = await consultarOdoo("res.users", "context_get");

        res.json({
            conectado: true,
            mensaje: "Conexión con Odoo 19 correcta",
            datos: resultado
        });

    } catch (error) {

        console.error("Error Odoo:", error.message);

        res.status(500).json({
            conectado: false,
            error: error.message
        });
    }
});

app.get("/api/odoo/clientes", async (req, res) => {

    try {
        const clientes = await consultarOdoo("res.partner", "search_read", {
            domain: [["customer_rank", ">", 0]],
            fields: ["id", "name", "email", "phone"],
            limit: 50
        });

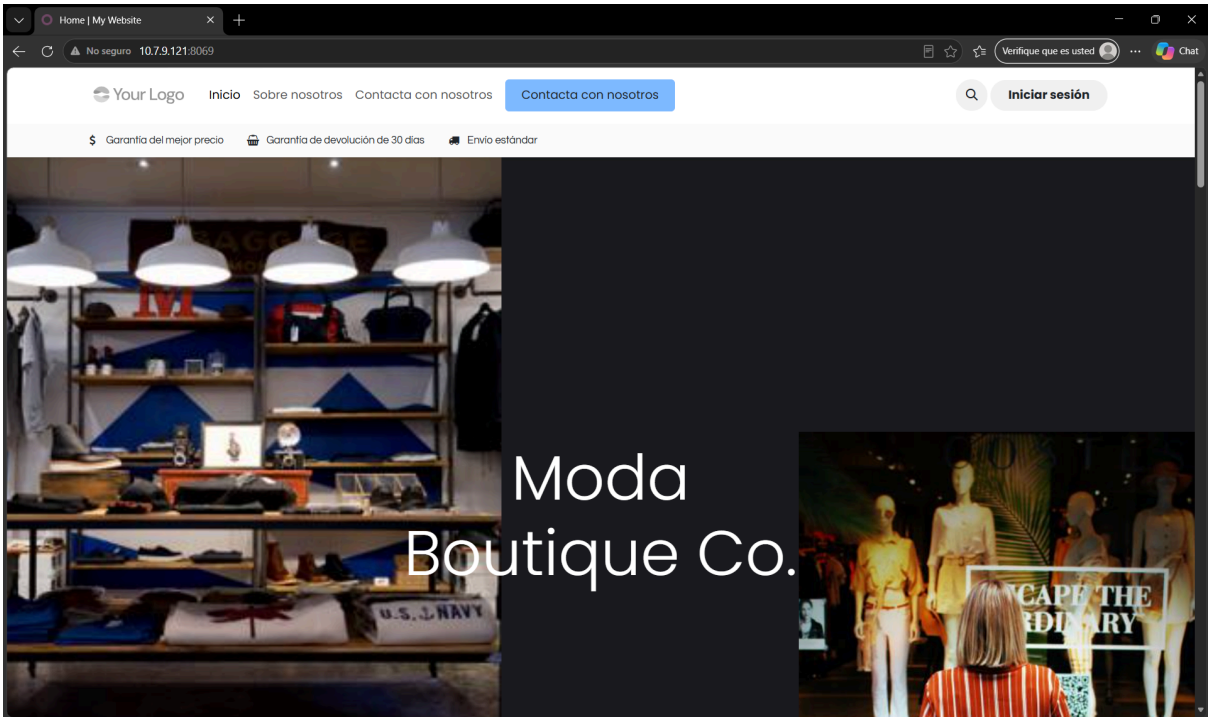
        res.json(clientes);

    } catch (error) {

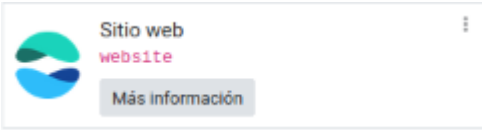
        console.error("Error clientes Odoo:", error.message);

        res.status(500).json({
            error: "No se pudieron obtener los clientes de Odoo"
        });
    }
});
```

Sin embargo, durante las pruebas de integración aparecieron varios problemas. En un primer intento, aunque los endpoints de consulta responden correctamente, al intentar enlazar la lógica de pedidos con la aplicación web se produjeron errores en el tiempo de ejecución que afectaron a parte de su funcionamiento. Tras corregir los fallos y volver a un estado funcional anterior, se llevó a cabo una segunda prueba. En esta ocasión ocurrió un conflicto distinto, al acceder al entorno web, en lugar de cargarse la interfaz desarrollada para el proyecto, el navegador mostraba la plantilla web predeterminada de Odoo.



Al revisar el gestor de aplicaciones del ERP, se comprobó que durante las pruebas se había activado automáticamente el módulo nativo de Sitio web, el cual estaba interceptando las rutas HTTP públicas y sirviendo su propia página de inicio. Se procedió a desinstalar dicho módulo desde Odoo para intentar recuperar la configuración previa, pero debido a las dependencias y vistas residuales que ya había generado en la instancia, su eliminación no resolvió el problema de integración.



Ante esta situación, y con el objetivo de asegurar la entrega del proyecto sin poner en riesgo el funcionamiento de los sistemas que ya estaban operativos, se decidió aplicar una solución alternativa. Esta consistió en realizar la transferencia de datos por lotes de forma manual mediante archivos en formato CSV. De este modo, los pedidos registrados en la base de datos externa de PostgreSQL se exportan y posteriormente se importan en Odoo a través de su herramienta nativa de importación.
Aunque esta alternativa sustituye la sincronización automática en tiempo real por un proceso manual controlado, permitió cumplir con los requisitos funcionales del proyecto y disponer de la información de los pedidos actualizada tanto en PostgreSQL como en Odoo de forma segura.

Capítulo 6. Conclusiones.

1. Conclusiones del trabajo

Como conclusión, consideramos que el desarrollo del proyecto ha sido satisfactorio, ya que ha permitido aplicar de forma práctica los conocimientos adquiridos a lo largo de la evaluación, además de ampliar dichos conocimientos de forma autodidacta mediante el uso de nuevas herramientas y tecnologías.

El proyecto ha integrado correctamente las distintas fases de análisis, diseño y desarrollo de una base de datos orientada a un sistema de gestión completo de una empresa. Se ha logrado que funcione internamente de forma automática mediante el uso de triggers, lo que permite introducir columnas como precios totales, añadir tablas como log eventos o comprobar que los el valor de algunos datos no pase cierto umbral.

La página web muestra una estructura avanzada utilizando diseño responsive para la visualización en distintos dispositivos y resoluciones de pantalla, Además de que mantiene la identidad visual entre las distintas pantallas y añade un sistema de personalización de zapatillas además de un sistema de inicio de sesión. Otra de las funciones principales es la conexión con la base de datos que permite a los clientes realizar pedidos y que estos se almacenen en la base de datos, así como la información de los clientes.

Por otro lado, el desarrollo del dashboard para análisis de datos mediante Python y Streamlit ha permitido transformar todos estos datos en información más clara y útil para la empresa. Gracias a él es posible consultar diversos indicadores y obtener información acerca de tendencias.

También es importante mencionar la implementación mediante Odoo, que ha permitido centralizar la administración de la empresa de forma gráfica. Gracias a él se puede gestionar de forma sencilla los distintos clientes, pedidos o productos. De esta forma se complementa con la base de datos y facilita la gestión del sistema empresarial.

Por último la aplicación desarrollada en Java permite abordar la gestión de los pedidos personalizados mediante la ejecución de varios procesos coordinados de forma simultánea. Gracias a esto se ha podido trabajar con conceptos relacionados con la concurrencia.

En conclusión, este proyecto nos ha permitido comprobar la importancia de integrar correctamente diferentes tecnologías para el uso adecuado del registro completo de sesión, usuario, gestión de pedidos entre otros.

Capítulo 7. Referencias.

Manual de Odoo: https://www.odoo.com/es_ES/page/docs

Fuentes de Letras: https://fonts.google.com

Imágenes de la Web: https://unsplash.com/es

Manual Express-Node: https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction